

NUS-ISS Intelligent Reasoning Systems

Practice Module Group Project Report

PropertyLens

Explainable AI for Singapore HDB Resale Price Transparency

Group Name

Transparent AI (Group 15)

Member	Student ID
KUMAR BHUVESH	A0243823H
LOU YUNCHUAN	A0340534M
CHI THRA REKHA	A0103372U

Submission Date

03 May 2026

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
AVM	Automated Valuation Model
BERT	Bidirectional Encoder Representations from Transformers
CBR	Case-Based Reasoning
CGS	Cognitive Systems
CLIP	Contrastive Language-Image Pretraining
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DOM	Document Object Model
HDB	Housing and Development Board
IRS	Intelligent Reasoning Systems
ISS	Institute of Systems Science
JSON	JavaScript Object Notation
JWT	JSON Web Token
KG	Knowledge Graph
LAION	Large-scale Artificial Intelligence Open Network (open image dataset used for aesthetic scoring)
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large Language Model
LRT	Light Rapid Transit
LSUN	Large-Scale Scene Understanding (image dataset)
MAE	Mean Absolute Error

MAPE	Mean Absolute Percentage Error
MCP	Model Context Protocol
ML	Machine Learning
MOE	Ministry of Education (Singapore)
MR	Machine Reasoning
MRT	Mass Rapid Transit
NUS	National University of Singapore
POI	Point of Interest
RAG	Retrieval-Augmented Generation
RF	Random Forest
RMSE	Root Mean Square Error
RS	Reasoning Systems
SGD	Singapore Dollar
SHAP	SHapley Additive exPlanations
SRX	Singapore Real Estate Exchange (property portal and valuation platform)
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
WSL	Windows Subsystem for Linux
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting

Acknowledgements

The team thanks the NUS-ISS IRS-PM module instructors for guidance and feedback throughout the project.

This work would not have been possible without the open data ecosystem provided by data.gov.sg, OneMap, OpenStreetMap, and the Ministry of Education.

AI coding assistants (Cursor, Claude) were used for code scaffolding, data exploration, and document drafting. All model design decisions, feature engineering rationale, and architectural choices were made by the team. AI tools served as productivity accelerators, not as decision makers.

Executive Summary

PropertyLens helps Singapore HDB resale buyers and sellers value and analyse flats, with explainability built into every step. The system combines a hybrid cluster stack regressor, four complementary explainability methods, a photo condition CNN, a Neo4j backed conversational interface, a React web application with eleven views, and a Chrome extension that bridges PropertyGuru into the valuation flow.

The hybrid model reaches 3.92% MAPE on a temporal hold out test set, a 68% reduction over a single XGBoost baseline of 12.18% MAPE. Composite TreeSHAP attributions are exact at machine precision across 100 validation cases. The chatbot was evaluated against two complementary test suites. A 25 case category cross validation achieved 80% strict pass on natural language search under stress, and an 11 case end to end capability validation achieved 100% pass across every chatbot branch including tool dispatch and shortlist composition. The photo CNN scores furnishing conditions within plus or minus 0.5 points on a held out validation set.

This report documents the business case, market positioning, system design, implementation approach, validation methodology, and measured findings. The system meets every functional and non-functional objective defined for the project.

Table of Contents

LIST OF ABBREVIATIONS	2
ACKNOWLEDGEMENTS	4
EXECUTIVE SUMMARY	5
1. BUSINESS CASE	8
1.1 THE BUSINESS CASE	8
1.2 THE PROBLEM	9
1.3 THE SOLUTION	10
1.4 WHY EXPLAINABILITY MATTERS	18
2. MARKET RESEARCH	19
2.1 THE COMPETITIVE LANDSCAPE	19
2.2 CAPABILITY COMPARISON	19
2.3 ACADEMIC PRECEDENT FOR THE DESIGN CHOICES	20
2.4 WHERE PROPERTYLENS DIFFERENTIATES	21
3. PROJECT OBJECTIVES AND SUCCESS MEASUREMENTS	22
3.1 FUNCTIONAL OBJECTIVES	22
3.2 NON-FUNCTIONAL OBJECTIVES	22
3.3 SUCCESS METRICS	23
3.4 OUT OF SCOPE	25
4. SYSTEM DESIGN AND MODEL	26
4.1 SYSTEM ARCHITECTURE OVERVIEW	26
4.2 DATA LAYER	27
4.3 FEATURE LAYER	28
4.4 ML LAYER, HYBRID CLUSTER STACK REGRESSOR	29
4.5 EXPLAINABILITY LAYER	33
4.6 PHOTO CONDITION CNN	37
4.7 KNOWLEDGE GRAPH AND ASK AI CHATBOT	38
4.8 WEB APP AND CHROME EXTENSION	41
5. SYSTEM DEVELOPMENT AND IMPLEMENTATION	43
5.1 TECH STACK AND RATIONALE	43
5.2 BACKEND ARCHITECTURE	44
5.3 FRONTEND ARCHITECTURE	45
5.4 CHROME EXTENSION IMPLEMENTATION	45
5.5 PERSISTENCE AND EXTERNAL SERVICES	46
5.6 REPRODUCIBILITY AND DEPLOYMENT	47
5.7 TESTING STRATEGY	47
6. FINDINGS AND DISCUSSION	49
6.1 HYBRID MODEL PERFORMANCE	49

6.2 CLUSTER LEVEL ANALYSIS.....	49
6.3 PHOTO CONDITION CNN RESULTS	50
6.4 EXPLAINABILITY LAYER BEHAVIOUR.....	51
6.5 CHATBOT EVALUATION	53
6.5.1 TESTING THE CHATBOT ON CROSS CATEGORY QUESTIONS	53
6.5.2 TESTING ALL CHATBOT CAPABILITIES VALIDATED END-TO-END AGAINST THE RUNNING SYSTEM.....	53
6.6 SYSTEM LEVEL VALIDATION AND LATENCY	54
6.7 WHAT SURPRISED THE TEAM.....	55
7. CONCLUSIONS, LIMITATIONS, AND FUTURE WORK	56
7.1 WHAT WE BUILT AND WHAT IT PROVES	56
7.2 LIMITATIONS.....	56
7.3 FUTURE WORK	57
7.4 CLOSING THOUGHTS.....	58
APPENDIX A. PROJECT PROPOSAL	60
APPENDIX B. MAPPED SYSTEM FUNCTIONALITIES AGAINST MR, RS, CGS.....	61
APPENDIX C. INSTALLATION AND USER GUIDE	63
APPENDIX D. CHATBOT EVALUATION, FULL PER CATEGORY TRACE.....	65
APPENDIX E: REFERENCES.....	69
APPENDIX F. COMPOSITE TREESHAP HELD-OUT VALIDATION.....	70
APPENDIX G. PINECONE VECTOR RAG, EXPLORATORY WORK	73

1. Business Case

1.1 The business case

The problems PropertyLens solves are universal in this market. Every HDB resale buyer and seller in Singapore faces them.

Market size

The HDB resale market is one of the largest segments of the Singapore housing landscape.

- **77% of residents live in HDB flats.** Public housing is the default tenure, not the exception.
- **25,000 to 30,000 transactions per year.** 28,986 in 2024 and 26,169 in 2025.
- Each transaction needs three artefacts: a defensible fair value, a negotiation plan, and a pricing strategy.

Today, none of these three artefacts exist outside the head of a property agent.

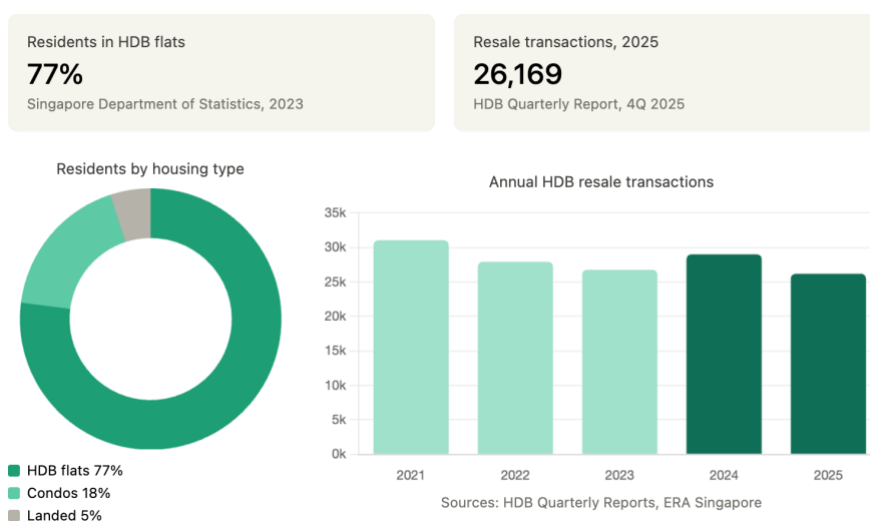


Figure 1: The visualization above shows the Market Size paragraph in three pieces.

Who pays, and for what

PropertyLens uses a freemium model with three tiers. Individual buyers and sellers can validate a single flat for free. Power users and agents pay a small subscription to unlock the full toolkit at scale.

- The buyer pays for confidence in the largest single purchase of their life. A buyer who avoids overpaying by even 1% on a S\$600,000 flat saves S\$6,000. That return is several hundred times the cost of three months on the Pro plan at S\$9.99 per month.
- The seller pays for asking price strategy and negotiation evidence. A seller who lists 2% closer to the right number sells faster and avoids the cost of a flat sitting on the market for an extra month, roughly S\$2,000 to S\$3,000 in mortgage interest and opportunity cost on a typical resale flat. The Pro subscription pays for itself within the first week of listing.
- Property agencies pay for the multi flat comparison, the SHAP driver charts, the Plan Your Offer card, and the white label branding, all of which they can hand to clients as branded analysis. The agent's pitch becomes a defensible argument backed by data rather than a request for trust on the comparables. The Business plan at S\$99 per month serves a single agent or a small team and scales through the API for larger agencies.

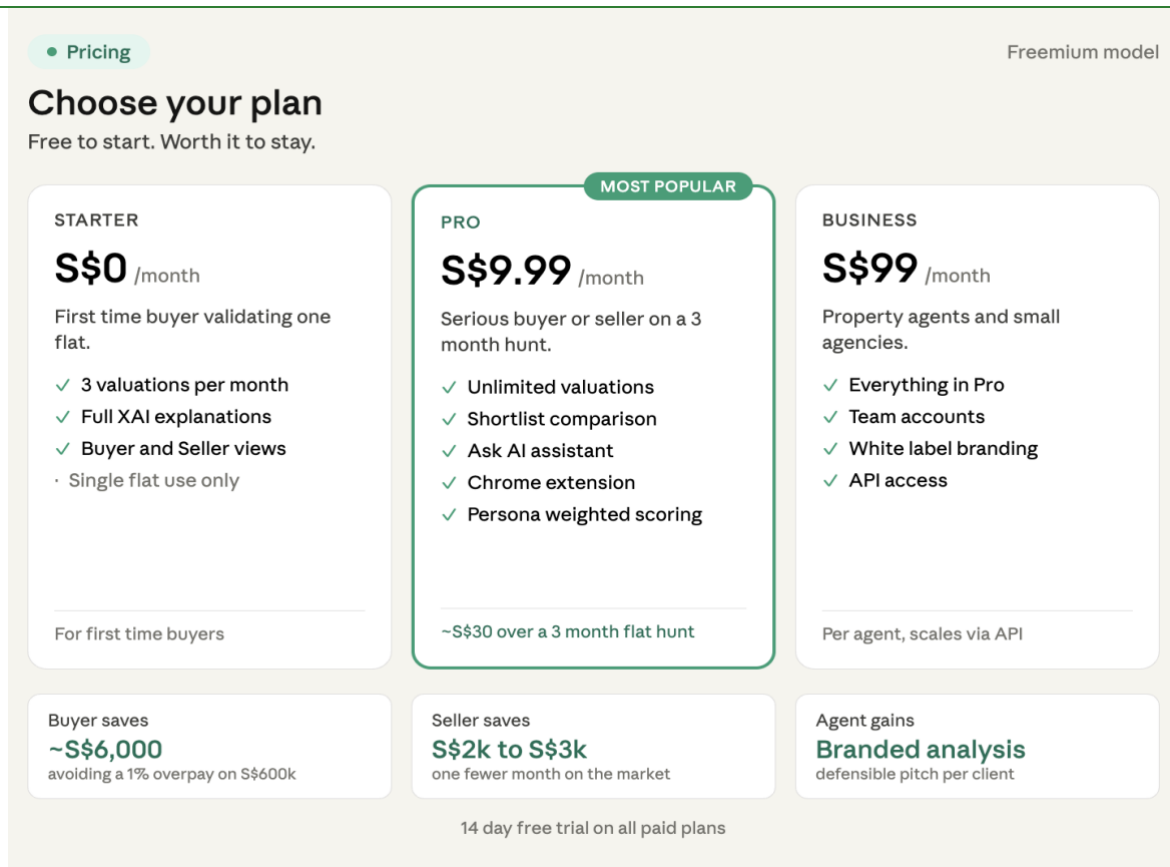


Figure 2: Pricing model

Pricing here is illustrative. The \$S\$9.99 and \$S\$99 figures position PropertyLens against comparable consumer and SaaS productivity tools rather than against measured willingness to pay.

Why now

Three forces make this the right moment for an explainable valuation tool.

- **Open data.** HDB transaction data is fully open through data.gov.sg, with monthly updates and complete coverage back to 1990. The raw material exists in a form that the major portals do not surface to end users.
- **Buyer expectations.** The same buyers who get itemised price breakdowns from Grab, Shopee, and every airline booking site now expect the same transparency from a \$S\$600,000 housing decision. PropertyGuru and 99.co have not updated their UX to match.
- **Mature technology stack.** SHAP for tree models, Neo4j for knowledge graphs, and small specialised CNNs for photo scoring are all production ready. PropertyLens integrates these pieces into a workflow that maps to how Singaporeans actually buy and sell HDB flats.

1.2 The problem

PropertyLens addresses two user types with mirrored problems on opposite sides of the same transaction.

The buyer's problem

A buyer browsing PropertyGuru finds a listing and immediately runs into three questions that the listing does not answer.

- **Is the asking price fair?** The seller's agent set that number. The buyer has no independent reference. The flat might be priced reasonably, or the asking price might sit 15% above what comparable flats actually sold for last quarter. PropertyGuru does not show closing prices, only other asking prices.
- **What drives this price?** The listing shows floor area, town, lease, and storey range. It does not show how each factor moves the price for this specific flat. Floor area might be lifting the value by S\$80,000. The remaining lease might be dragging it down by S\$45,000. The buyer cannot see this breakdown anywhere.
- **How do I negotiate?** Even a buyer who arrives at a fair value figure still has no offer plan. The buyer needs a defensible opening offer, a walk away number, and the evidence to support each.

A **second pain** hits buyers who have shortlisted four or five flats. They now need to compare those flats on schools, MRT proximity, hawker centres, malls, and price per sqm. PropertyGuru does not place these side by side. The buyer ends up with five browser tabs, a notepad, and a mental model that breaks down by the third flat.

The seller's problem

The seller's problem is the inverse, with a twist.

The seller wants an asking price that earns the best return without sitting unsold for months. The naive approach of matching what neighbours are asking propagates over pricing across the development. PropertyGuru shows asking prices, not closing prices. data.gov.sg publishes closing prices in a CSV that the seller cannot easily interpret for their own flat without filtering and adjustment.

A seller who arrives at a defensible asking price still needs the next step. They need to know which features of this flat are strongest, which to highlight in the listing, and which are weaknesses to address in negotiation.

1.3 The solution

PropertyLens splits the user workflow into three views, each tied to one of the problems above. A Chrome extension bridges PropertyGuru into PropertyLens.

Buyer view

The buyer keeps browsing PropertyGuru. When they see a flat they like, they click "Open in PropertyLens" from the Chrome extension. The flat opens in the Buyer view with the listing fields filled in automatically.

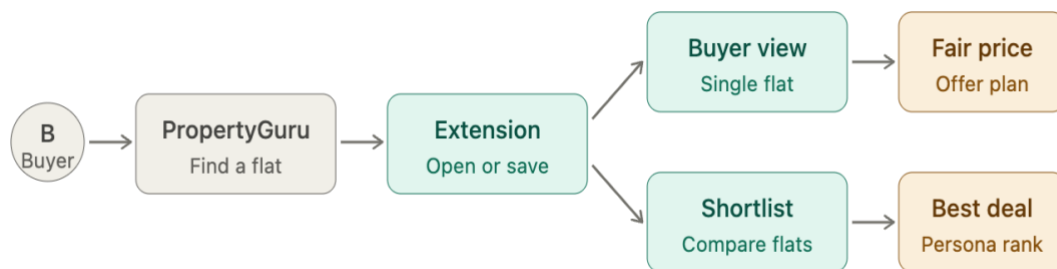


Figure 3: Buyer's journey

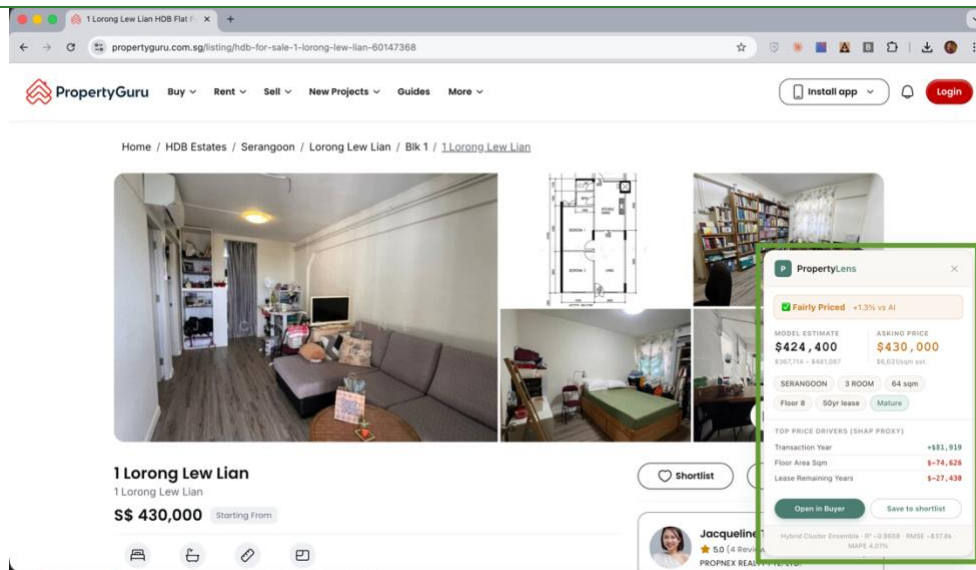


Figure 4: Buyer looking at a property on PropertyGuru. The Buyer clicks on 'Open in Buyer' button on the extension.

The view answers the three questions the listing did not.

- It shows an AI fair value estimate with a confidence band. Figure below shows an asking price of S\$430,000 sitting within 1.5% of the AI estimate of S\$423,600, flagged as a fair price.

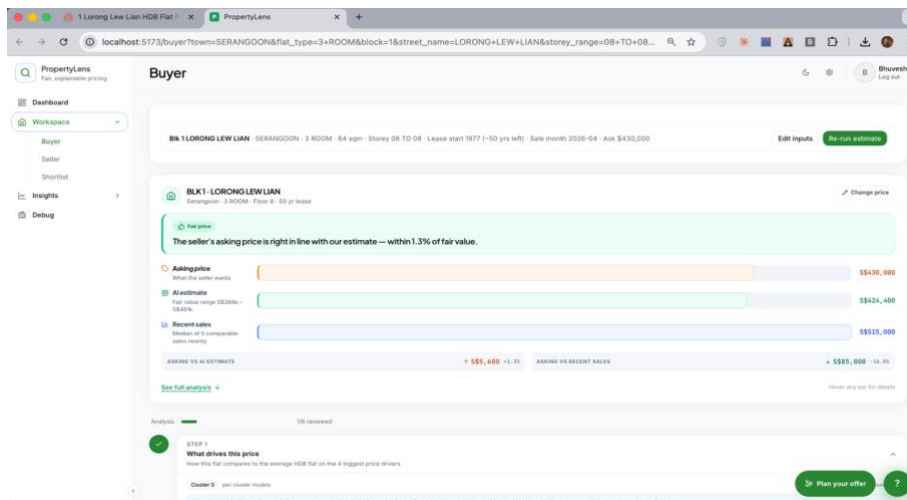


Figure 5: Buyer is then moved to Buyer's view on PropertyLens. Here the Buyer can see the AI estimated price for the listing.

- It shows a SHAP attribution chart that decomposes the predicted price into its top drivers. The Figure below shows market timing lifting price by S\$90,017, floor size dragging it down by S\$73,484, remaining lease dragging by S\$46,665, and so on across the top six factors and a rollup of 72 smaller features.

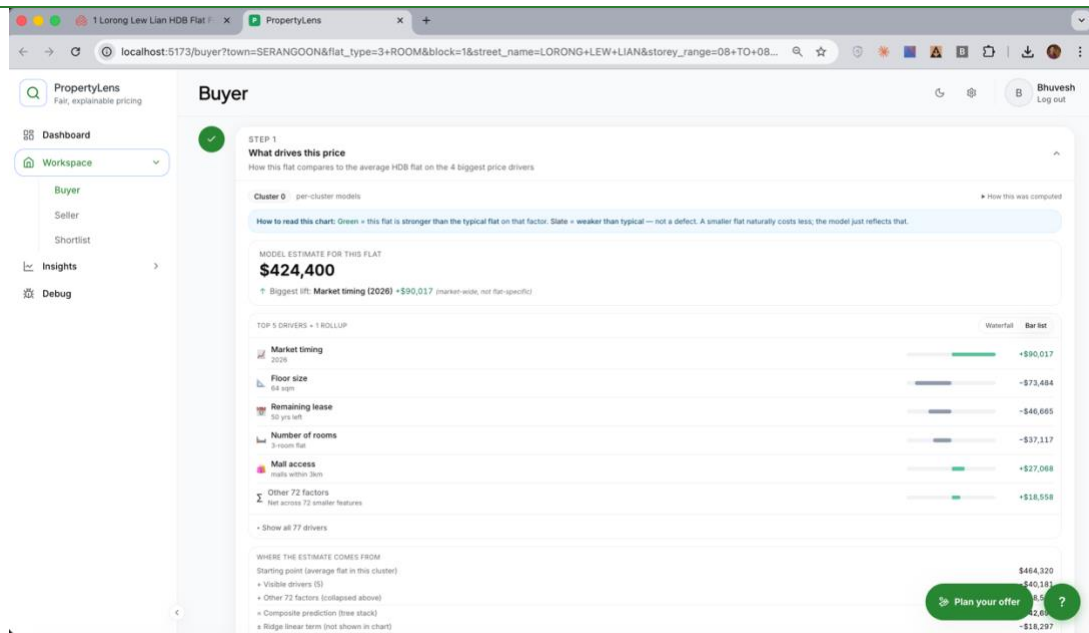


Figure 6: The buyer can scroll down and see different UI cards that help with further explanation of the AI estimate. One of such card is 'What drives the price' (that is backed up by SHAP values).

- It shows a what if tool that lets the buyer change attributes and rerun the model, a Plan Your Offer card that walks the buyer through walk away, opening, and fair offer numbers with the SHAP driver argument behind each, and a Case Based Reasoning panel with five comparable past sales drawn from the same neighbourhood.

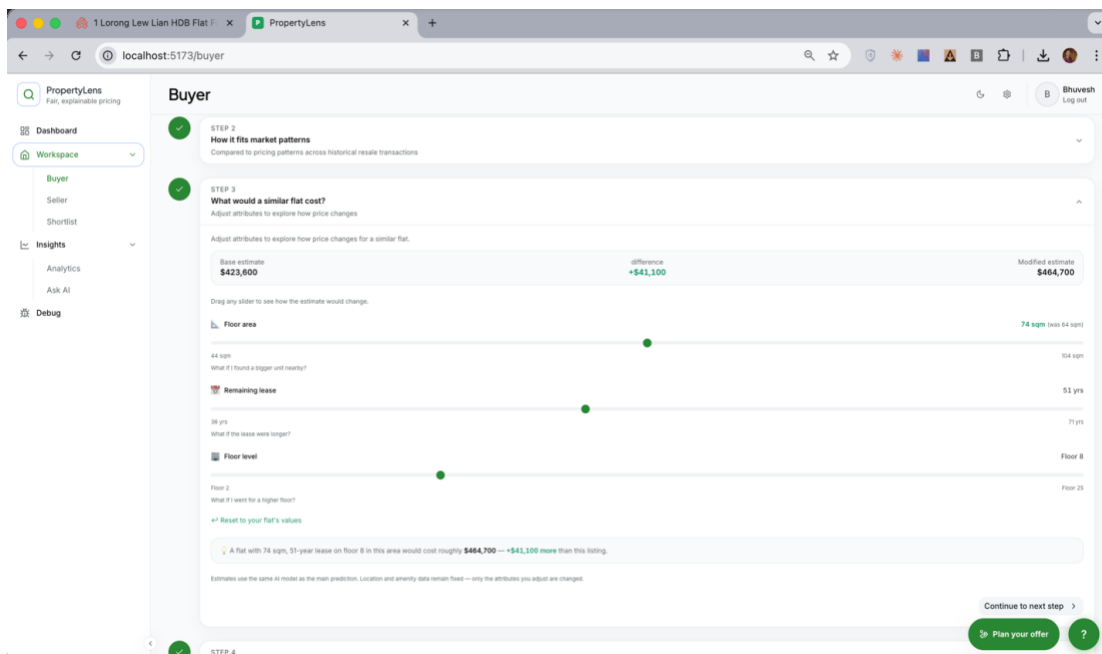


Figure 7: What-if tool on Buyer's view

STEP 5
Your negotiation strategy
Combined view from model, comparables, and patterns

ASKING PRICE: S\$430,000 AI FAIR VALUE: S\$423,600 LISTING BIAS: +1.5% (above target)

YOUR PLANNED OFFER
S\$ 412,000

SOLID OFFER S\$412,000 -4.2% vs ask -2.7% vs AI
At or below fair value. Anchor on the comps + AI estimate; stay calm if the seller counters.

Your three-number plan
Anchored on AI fair value and recent comps. Asking caps the walk-away (you never pay above the listed price).

Offer Type	Value	Description
Opening offer	S\$402,400	Anchored on AI fair value - 5% and the median of recent recent comps. Sellers typically counter +2-5% — leaves room.
Target close	S\$423,600	AI fair value, capped at the comp band + asking. You'd be happy to close here.
Walk-away ceiling	S\$430,000	Higher of AI + 1.03 and comps median + 1.05, capped at asking. Above this you're paying more than evidence supports.

Evidence to anchor your offer

- AI fair value: S\$423,600. Top driver: floor size.
- Median of 5 recent comparable sales is S\$515,000 (5 in-town).
- SERANGOON CTRL sold at S\$560,000 (2026).
- Listing is 1.5% above the AI band — useful to cite when negotiating down.

When the seller pushes back

"My asking is firm — that's what the market supports."
→ The AI fair value is S\$423,600; recent comps median is S\$515,000. My offer reflects what comparable flats actually sold at.

"There are multiple interested buyers."
→ That may be true, but my walk-away is S\$430,000. Above that I'd be paying more than the evidence supports.

"It's newly renovated."
→ I'll need to see the renovation receipts. Standard depreciation caps the recoverable premium at roughly 30% of cost — happy to revise if it checks out.

"The lease isn't a problem."
→ 51 years remaining is below the 70-year mark — bank LTV starts to taper, and the AI fair value already prices the lease decay in.

Continue to next step >

STEP 6
Location and amenities
What's within walking distance

Plan your offer ?

Figure 8: Plan your offer card on Buyer's view

- The buyer can also upload a photo of the flat's interior. A small CNN scores renovation conditions on a 0 to 10 scale and applies a price adjustment of up to plus or minus 10%. Figure below shows a condition score of 5.29 lifting the estimate by 0.6% to S\$426,904.

PropertyLens
Fair, explainable pricing

Buyer
Adjust attributes to explore how price changes.

STEP 4
What similar flats sold for
Comparable past transactions for context

STEP 5
Your negotiation strategy
Combined view from model, comparables, and patterns

STEP 6
Location and amenities
What's within walking distance

Verify the quality of renovation and see if it changes the price
Upload one interior photo (living room, kitchen, bedroom). We score its furnishing condition and apply a +10% adjustment — a well-presented flat can lift your estimate by tens of thousands.

Condition score: 5.29 / 10

Average: +0.6%

Base price: \$424,488
\$426,904
Adjusted price

Reset

Model: EfficientNet-B0 - val MAE: 0.49 - trained 20260413

Plan your offer ?

Figure 9: The chrome extension also extracts listing image, that can additionally help buyer to gauge the quality of the renovation.

Seller view

The Seller view answers the seller's question of what asking price to set. Figure below shows the seller's workspace for Blk 201 Tampines St 21. The model estimates the flat at S\$752,800 with a price per sqm of S\$8,183 and Medium confidence.

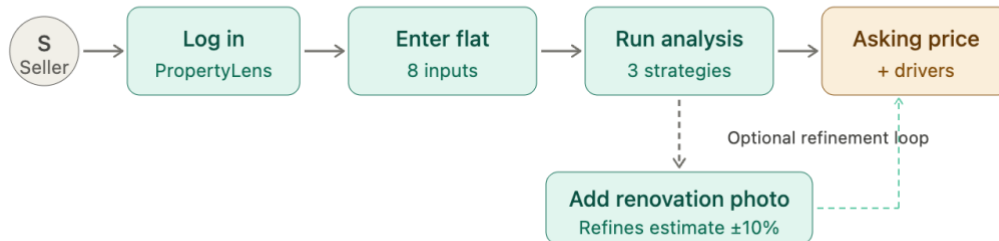


Figure 10: Seller' journey

The screenshot shows the 'Seller's workspace' interface. The left sidebar contains navigation links: Dashboard, Workspace (selected), Buyer, Seller, Shortlist, Insights, and Debug. The main content area is titled 'Seller's workspace' and includes a subtitle 'Fair market value, asking price guidance, and what drives your valuation.' Below this, there is a note: 'Example flat pre-filled (Blk 201 Tampines St 21) — edit any field to your own.' The form fields are as follows:

- BLOCK NUMBER: 201
- STREET NAME: TAMPINES ST 21
- TOWN: TAMPINES
- FLAT TYPE: 4 ROOM
- FLOOR AREA (SQM): 92
- SALE MONTH (VALUATION): April 2026
- STOREY RANGE: 01-03, 04-06, 07-09 (selected), 10-12, 13-15, 16-18, 19-21, 22-24, 25-27, 28-30, 31-33, 34-36, 37-39, 40+
- LEASE START YEAR: 1993
- REMAINING LEASE: 66 years remaining

Below the form fields, there is a note: 'Auto-calculated from lease start 1993 and sale month 2026-04.' At the bottom, there is an 'Advanced: manual distance overrides' section with a '+' icon. A green button labeled 'Analyse my flat' is at the bottom left, and a green question mark icon is at the bottom right.

Figure 11: Seller can enter details of their property as the Seller view page.

Three pricing strategies sit below the estimate.

- **Price to sell fast at S\$745,272.** Two to four weeks on the market.
- **Balance price and time at S\$775,384, the recommended option.** Four to eight weeks on the market.
- **Hold out at S\$801,392.** Two to four months on the market.

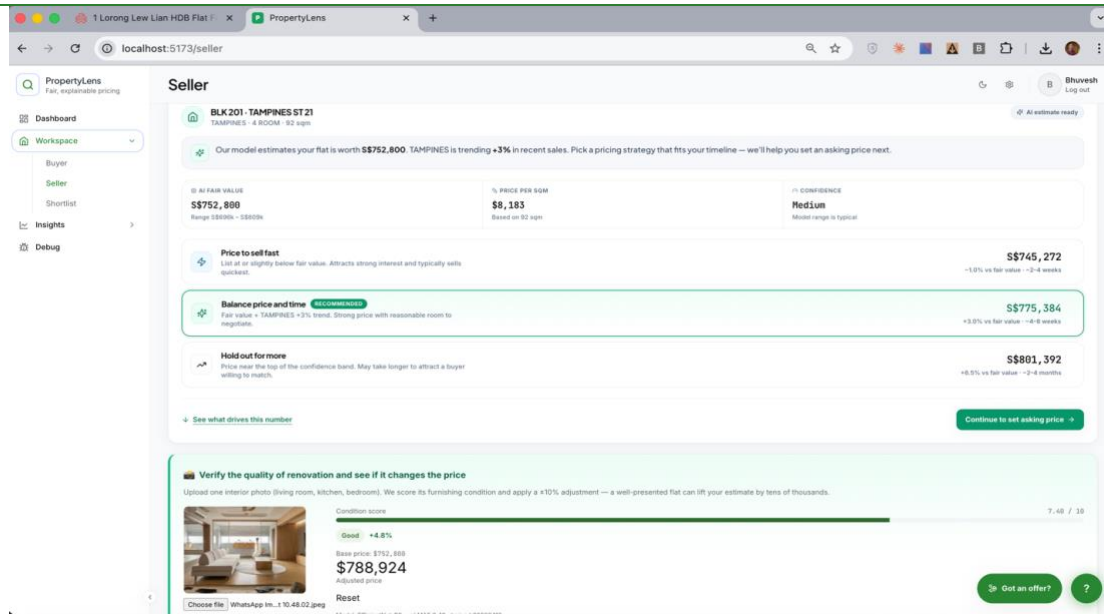


Figure 12: Seller then can view the valuation of their property.

The seller gets the same drivers chart as the buyer so they know which features to emphasise in the listing and which to defend during negotiation. A "got an offer" verdict lets them paste an incoming offer and see whether it sits inside the fair range. Photo scoring works the same way as in the Buyer view.

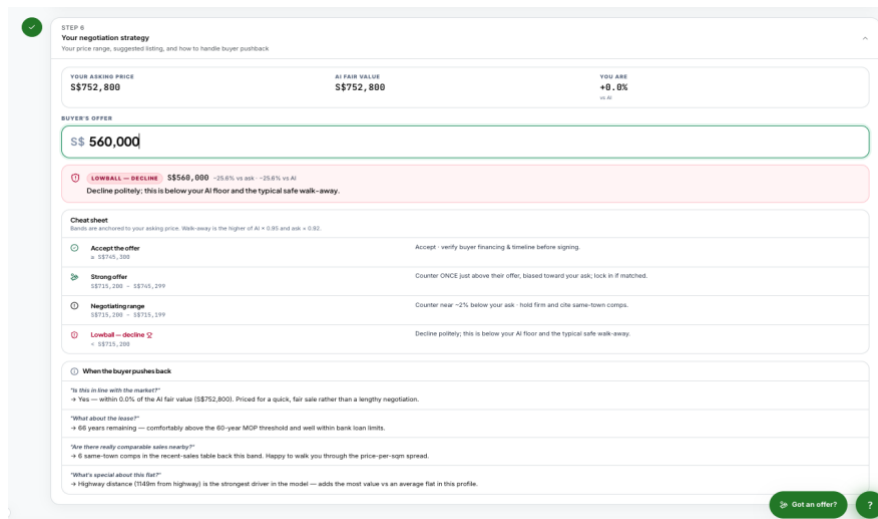


Figure 13: Get an Offer card on Seller's view

Shortlist view

For buyers comparing multiple flats, the Shortlist view solves the tab switching problem.

The buyer adds PropertyGuru listings to the shortlist through the Chrome extension. Figure below shows the saved flats ranked by Smart Score and flagged as Fair, Strong Deal, or Stretch based on how the asking price compares with the AI estimate. Three persona buttons reorder the list. Family weights schools, Commuter weights MRT and LRT proximity, and Investor weights deal quality from Smart Score.

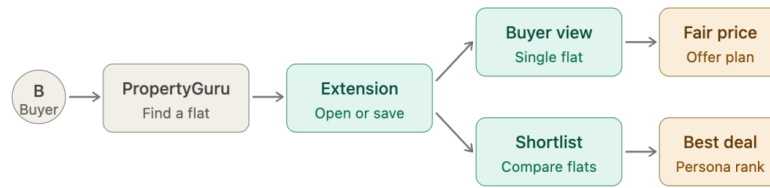


Figure 14: Shortlist view journey

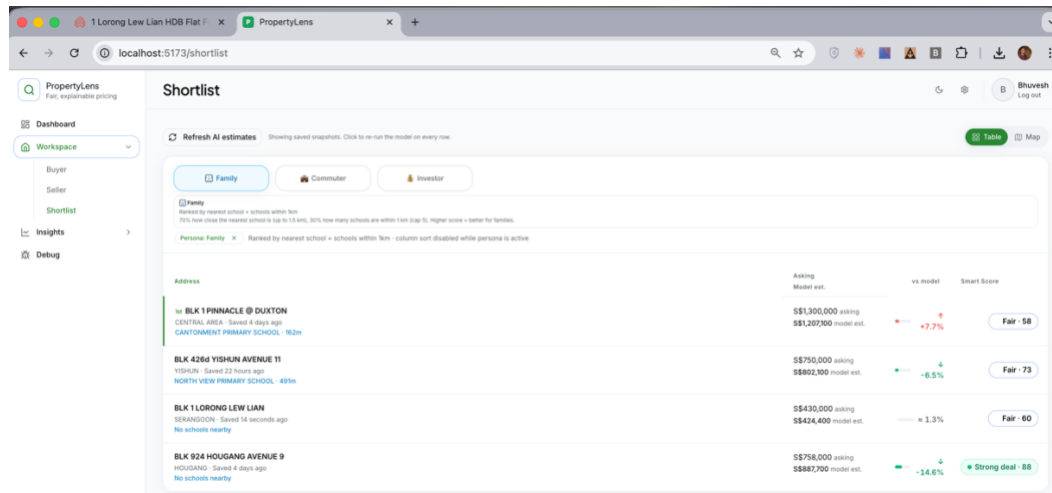


Figure 15: Shortlisting view showing all the shortlisted listings, along with three personas

The Compare view, shown in Figure below lets the buyer overlay up to three flats at once on price, schools, transport, and lifestyle amenities. A trophy icon marks the per row winner. The map view places all saved flats on the Singapore map with their walking radii for amenities, so the buyer sees the geographic context at one glance.

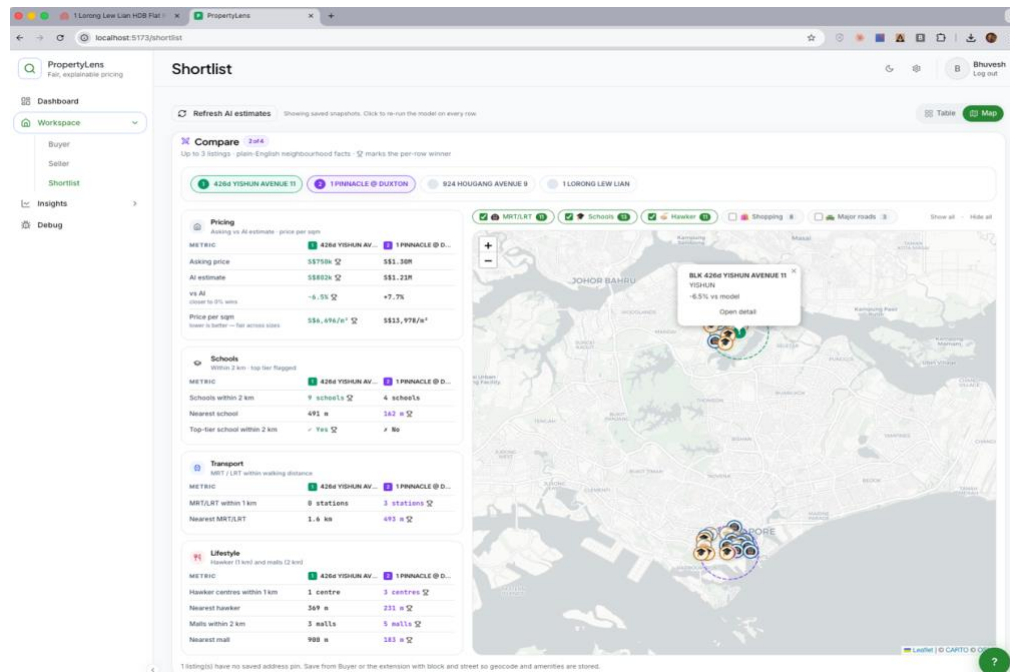


Figure 16: Shortlist view showing comparison between two listings, in cards and map view.

Ask AI

The Ask AI panel lets the user type a natural language query such as "Find a 4 room flat in Bishan near famous schools under \$900k". The pipeline runs three steps:

- Extracts structured parameters from the query.
- Runs a parameterised Cypher query against the Neo4j knowledge graph.
- Returns matching past transactions with a short plain English narrative.

The inferred parameters and applied weights appear in a debug panel so the user can audit exactly what the system searched for.

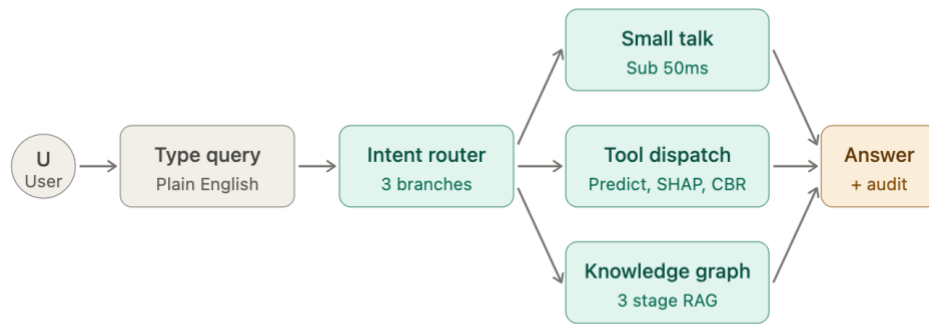


Figure 17: Ask AI conversational interface with auditable parameter extraction

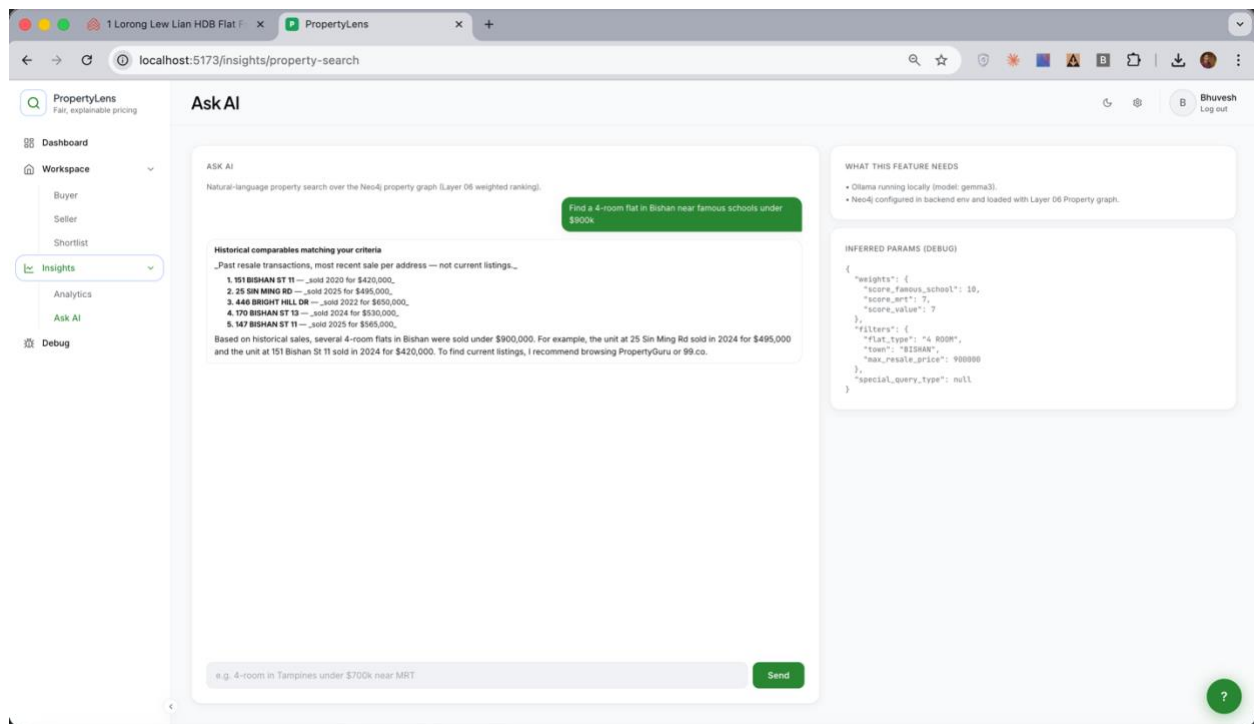


Figure 18: Showing the Ask AI view

1.4 Why explainability matters

The team could have built an opaque regressor that predicts HDB resale prices to single digit accuracy. The team chose not to. A black box price recommendation does not work in this market, for three reasons.

The buyer needs to defend the offer.

A buyer who tells their agent "the AI says S\$620,000" without an explanation loses the conversation. A buyer who can say "the AI says S\$620,000 because the floor area at 122 sqm is exceptional for this price band, partially offset by the 60 year remaining lease" has a position. SHAP gives them that breakdown for every prediction.

The seller needs to test trade-offs.

The right asking price depends on whether the seller wants to close fast or hold out for the top of the band. The three pricing strategy cards make the trade-off explicit instead of treating the AI estimate as a single fixed number.

The market expects auditability.

Singapore's housing market is heavily regulated. Any tool that influences pricing decisions at scale will eventually be examined for fairness, bias, and consistency. PropertyLens loads the same model bundle at startup, runs no online retraining, and exposes its inferred parameters and retrieved rows. A grader, an auditor, or a journalist can reproduce any prediction.

These three concerns make explainability a product requirement, not a feature.

2. Market Research

2.1 The competitive landscape

The Singapore HDB resale market is served today by three categories of tools. Listing portals, AI valuation tools, and academic projects.

Listing portals (PropertyGuru, 99.co, SRX)

PropertyGuru and 99.co are inventory layers, not valuation tools.

- Host listings, support filtering on basic attributes, surface comparable transactions on request.
- Show asking prices set by sellers and agents, not actual closing prices. Buyers calibrate against other asking prices, not real outcomes.
- The closest thing to a valuation is the comparables sidebar, which lists past sales for the same block or street without adjusting for floor, lease, or attribute differences.

AI valuation tools (SRX X Value, HomerAI)

SRX X Value is the longest running AI valuation product for Singapore residential property. It claims roughly 98% accuracy on its own test set. The output is a single point estimate with no breakdown of why the model arrived at that figure. The methodology is proprietary and the user cannot audit any prediction. SRX explicitly notes that X Value does not account for qualitative factors like unit interior condition.

HomerAI is the closest analogue to PropertyLens on capability scope. It produces an AI valuation, supports a chatbot called HomerGPT, and lets the user ask property questions in natural language. The chatbot focuses on valuation and proceeds calculation. It does not show per feature attribution. It does not surface comparable past sales as a structured panel. It does not let the user shortlist multiple flats and run side by side comparison with persona based scoring.

Academic and open source projects

GitHub hosts a steady stream of HDB price prediction projects, mostly notebooks. The original proposal cited four representative ones.

- Teyang lau ships a Streamlit app with an amenity map and Random Forest feature importance.
- JackFongNew uses Gradient Boosting.
- Hengbl uses amenity distance features.
- A Medium series by Nate Lee uses linear regression for the explainability that comes from coefficients.

These projects do well on raw accuracy. They stop at global feature importance, which is a single bar chart of what mattered across the entire dataset. They do not produce per listing explanations. They do not have a chatbot, a Chrome extension, or a deployed user facing system. Most are training notebooks that no buyer or seller could actually use.

2.2 Capability comparison

The table below maps PropertyLens against the four categories on nine capability dimensions. The dimensions are drawn from the original project proposal and updated against what the system actually shipped. Cells use short descriptors.

Capability	PropertyGuru	99.co / SRX	HomerAI	GitHub	PropertyLens
Price estimation	Comparables only	ML AVM	ML model	XGBoost / RF	Hybrid cluster stack

Capability	PropertyGuru	99.co / SRX	HomerAI	GitHub	PropertyLens
Per feature explanation	None	None	None	Global only	TreeSHAP, CBR, rules
Rich derived features	Basic	Basic	Basic	Varies	77 features
Neighbourhood signal	None	None	None	None	K means cluster routing
Chatbot	None	None	HomerGPT	None	Neo4j RAG + tools
Search and explanation	Filters only	Filters + comps	Match feature	None	Auditable Cypher
Rule extraction	None	None	None	None	Apriori + surrogate tree
Ground truth	Asking price	Transactions	Mixed	Varies	Gov transactions only
Methodology	Proprietary	Proprietary	Proprietary	Open source	Open, reproducible

PropertyLens is the only system in the table that combines per listing attribution, neighbourhood aware modelling, rule extraction, an auditable knowledge graph chatbot, and an open methodology. Each of the other categories wins on one or two dimensions. None of them combine the dimensions that matter for a real transaction conversation.

2.3 Academic precedent for the design choices

Three threads of academic work informed the system design.

Hybrid cluster stack architectures for heterogeneous markets

Monolithic regressors fail on heterogeneous markets.

- Standard regression trees, gradient boosted models, and neural network regressors treat the national market as a single distribution.
- Sub markets behave differently. Central premium flats, mid-range mature estates, and larger suburban non-mature flats follow different price drivers.
- Research on cluster-then-stack approaches (unsupervised routing followed by per-cluster specialised regressors) shows consistent error reductions on heterogeneous real estate datasets.

PropertyLens takes inspiration from academic Paper “A Hybrid Machine Learning Model Architecture with Clustering Analysis and Stacking Ensemble for Real Estate Price Prediction”

https://www.researchgate.net/publication/383237853_A_Hybrid_Machine_Learning_Model_Architecture_with_Clustering_Analysis_and_Stacking_Ensemble_for_Real_Estate_Price_Prediction and implements this in our context. A K means router on numeric routing dimensions partitions training data into three clusters. Each cluster has its own ensemble of Ridge, XGBoost, LightGBM, and Random Forest, blended by a Ridge meta learner. The architecture choice is what gets the model from a 6.81% MAPE single stack baseline to a 3.92% hybrid result on the temporal hold out test set.

Explainable AI for tabular regression

The XAI literature over the past five years has produced a small set of methods that became canonical for tabular regression.

- **SHAP** from Lundberg and Lee in 2017 for feature attribution, with TreeSHAP as the exact polynomial time variant for tree models.

- **Apriori with surrogate rules**, building on Agrawal and Srikant in 1994, for mining frequent patterns and pairing them with an interpretable surrogate model for global rule extraction
- **Case Based Reasoning**, formalised by Aamodt and Plaza in 1994, for retrieval of analogous training examples.
- **Counterfactual explanations** for "what would have to change" reasoning.

PropertyLens uses four of these. Composite TreeSHAP, because the hybrid stack mixes tree and non-tree learners and a single TreeSHAP call is not exact for the full stack. CBR via a BallTree of past transactions. Counterfactual scenario generation. Apriori mined surrogate rules for global pattern surfacing. LIME is implemented in the backend but not currently surfaced in the production frontend.

Knowledge graphs for explainable retrieval

Vector RAG dominated industry RAG implementations over the past two years but is opaque. Retrieved chunks are similar by some learned distance metric, but the user cannot audit why a specific result surfaced. Knowledge graph grounded retrieval preserves auditability at the cost of a curated schema.

PropertyLens deliberately chose the graph path for the chatbot.

- **Backing store.** Neo4j knowledge graph behind a 3-stage RAG pipeline.
- **Pipeline.** Natural language → parameterised Cypher → structured results → LLM 2 to 3 sentence narrative.
- **Audit surface.** Inferred parameters and retrieved rows are exposed in the UI.

The team also evaluated a Pinecone vector RAG implementation (preserved in the exploratory notebooks) and rejected it because the graph exposes the inferred parameters and retrieved rows for user audit.

2.4 Where PropertyLens differentiates

PropertyLens is positioned relative to the four categories above as follows.

- **Versus PropertyGuru and 99.co**, PropertyLens is complementary, not competitive. The Chrome extension is built specifically to deep link from a PropertyGuru listing page into the PropertyLens Buyer view. Discovery stays on PropertyGuru. Valuation and explanation move to PropertyLens. The two systems are designed to be used together.
- **Versus SRX X Value and HomerAI**, PropertyLens is a direct comparison point. PropertyLens commits to per prediction explanation through four user facing XAI methods. It returns prediction bands, not single point values. It surfaces comparable past sales through Case Based Reasoning. It supports counterfactual scenario planning. The hybrid model reaches MAPE 3.92% and R squared 0.9644 on a temporal hold out test set covering transactions from January 2025 onward.
- **Versus raw data.gov.sg**, PropertyLens is the user facing layer on top. The model trains on data.gov.sg directly. The user interacts with the model.
- **Versus academic and GitHub projects**, PropertyLens is an end to end deployed system with a frontend, a backend, a chatbot, a browser extension, and a reproducible artefact pipeline.

The Shortlist view is the most distinctive PropertyLens capability. No listing portal or valuation tool supports anything equivalent.

A user with five saved flats sees, in one screen:

- Which flat is the best deal on which dimensions.
- Which fits a family persona versus a commuter persona.
- Where the saved flats overlap in walkable amenities.

3. Project Objectives and Success Measurements

3.1 Functional objectives

PropertyLens was scoped against six functional objectives. Each objective ties to a specific user facing capability and a corresponding subsystem in the codebase.

ID	Objective	Description
F1	Predict fair value	The user supplies eight inputs (block, street, town, flat type, floor area, storey range, lease year, transaction year). The system returns a point estimate, a prediction band, and the inferred cluster assignment. Implemented through the hybrid cluster stack loaded from a serialised model bundle at backend startup.
F2	Explain every prediction	Four user facing explanation methods, each addressing a different question. • Composite TreeSHAP for feature attribution. • Case Based Reasoning for analogous past transactions. • Counterfactual scenarios for negotiation range planning. • Apriori surrogate rules for global pattern context.
F3	Three workflows with map	Buyer, Seller, Shortlist views. Each consumes the same backend but presents it in the frame of the user's job to be done. Map panels in Buyer and Seller views and the Shortlist Compare view show flats with nearby MRT, schools, hawker centres, and malls within walking radii.
F4	Auditable conversational interface	Plain English query. LLM extracts structured parameters. Parameterised Cypher query against Neo4j. Natural language narrative grounded in retrieved rows. Both inferred parameters and retrieved rows exposed for user audit.
F5	Bridge from PropertyGuru	Chrome extension scrapes listing fields from PropertyGuru pages and pre fills them in the Buyer view. Discovery stays on PropertyGuru. Valuation moves to PropertyLens.
F6	Estimate renovation quality	User uploads one interior photo. EfficientNet B0 scores furnishing condition on a 0 to 10 scale. Rule based price adjustment of up to plus or minus 10% on the base estimate.

3.2 Non-functional objectives

Four non-functional concerns shaped the architecture as much as the functional requirements did. Each one drove a concrete design decision, and each one is testable.

ID	Concern	Design impact
N1	Latency budgets per branch for chatbot	Three explicit budgets. Small talk and intent recognised tool dispatch on the fast path with sub 50ms response. Prediction and SHAP attribution sub second once the bundle is in memory. Full RAG natural language search 5 to 8 seconds, dominated by Ollama warm state latency.
N2	Explainability depth	Per prediction explanation, not just global. Every prediction comes with its own SHAP decomposition, CBR neighbourhood, and counterfactual scenarios. Computed on the fly per request rather than cached.
N3	Auditability	Every artefact reproducible from project notebooks. Every model artefact carries a version manifest. Every chatbot response exposes inferred Cypher parameters and retrieved rows for user inspection.

ID	Concern	Design impact
N4	Smooth flow inventory to valuation	Chrome extension removes copy paste. Listing fields scraped from PropertyGuru DOM and pre filled in Buyer view. Web app and extension share session state through the same JWT. Deep links into Buyer, Shortlist, and Compare views carry listing context as URL parameters.

3.3 Success metrics

Each objective above has one or more quantitative metrics the system is graded on. The full numbers are reported in Section 6. This section establishes what we measure, how we measure it, and why.

M1. Model accuracy

Measured on a temporal hold out test set. Training data is transactions before 2024. Validation is 2024 transactions. The test set is transactions from January 2025 onward. The model is graded on its ability to predict transactions it has never seen and whose date is later than any training point.

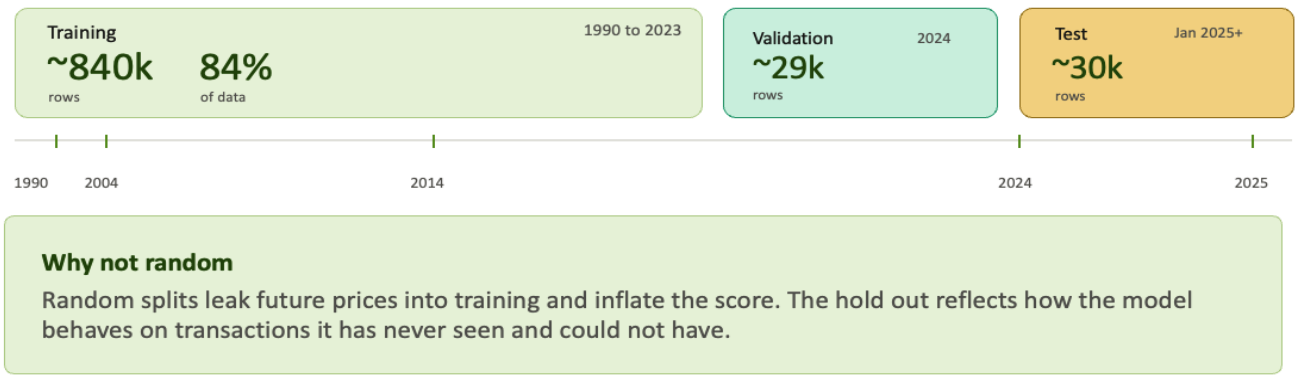


Figure 19: Temporal Split

Metric	Hybrid model	Stacked ensemble on features, no clustering	XGBoost on features, single stack
MAPE	3.92%	6.21%	12.18%
R squared	0.9644	0.9303	0.7972
RMSE	\$38,549	\$53,243	\$90,822
MAE	\$26,104	\$41,172	\$79,757

The 68% MAPE reduction over the XGBoost baseline is the headline architectural justification.

M2. Chatbot evaluation

The chatbot was evaluated against two complementary test suites that probe different aspects of the system. Together they exercise every chatbot capability the production deployment exposes.

Suite	Case	What it tests
Suite 1, Category cross validation	25	Stress test of the natural language search branch alone. Six categories of weighted Cypher queries with intentionally hard combinations including impossible queries. Strict PASS / PARTIAL / FAIL grading on parameter extraction, retrieval correctness, and narrative grounding.
Suite 2, End to end capability validation	11	Verifies every chatbot capability runs end to end. Greeting fast path, weighted RAG, knowledge graph traversal, honest framing for out of distribution queries, tool dispatch (predict, CBR, SHAP).

More details are provided in Appendix D.

M3. Photo condition CNN

EfficientNet B0 fine tuned on LSUN bedroom photos with LAION aesthetic scores. Validation MAE is 0.483 on a 0 to 10 scale. The model gets within plus or minus 0.5 points of human annotated furnishing condition on held out images. The price adjustment is bounded at plus or minus 10% so the CNN cannot dominate the hybrid model estimate.

M4. End to end system validation

Smoke test. Every documented user flow must complete without error. Sign in, predict, save to shortlist, compare, ask AI. Latency must hit each of the three budgets in N1. The flow is exercised manually before each commit to the main branch.

M5. Raw data and feature validation

The raw dataset has schema differences across periods, missing remaining lease columns, town name inconsistencies, and imperfect OneMap geocoding.

Three explicit validation checks run on every pipeline build.

- **Schema consistency check.** Every input CSV is loaded through a strict schema validator that asserts column types and ranges. Resale price between \$50,000 and \$5,000,000. Floor area between 30 and 250 sqm. Storey range in the documented bucket set. Failures are logged and offending rows dropped. On the final training cut, 0.4% of rows fail at least one schema check and are dropped.
- **Geocoding spot check.** After OneMap geocoding completes, a random sample of 200 blocks is manually verified against Google Maps. Current spot check rate is 96.5% correct, with the remaining 3.5% mostly resolved within 100 metres of the actual centroid. Blocks resolving more than 500 metres away are flagged and manually corrected.
- **Feature engineering sanity check.** After amenity distance and count features are computed, distributions are inspected against expected ranges. MRT walk distance between 0 and 5,000 metres for over 99% of flats. Schools within 2km between 0 and 25. Hawker count within 1km between 0 and 8. Outliers fixed at source or dropped.

M6. XAI validation through held out test cases

The team validated each XAI method through a fixed set of 100 test cases drawn from the temporal hold out test set. None of the 100 cases appear in the training data. The 100 cases were stratified across the three K means clusters and the four major flat types, so every cluster and type combination has at least 6 cases. The validation is reproducible with seed 42.

Property verified	Result	Detail
Composite TreeSHAP additivity	100 / 100	Median error 0.00003%, worst 0.00014%. Machine precision. The 77 attributions plus cluster baseline reconstruct the tree only stack prediction exactly.
SHAP determinism	100 / 100	Top 3 drivers identical across 5 reruns. Absolute attribution values match to 4 decimal places. Expected for an exact tree explainer.
SHAP domain plausibility	95 / 100	5 cluster 1, 5 ROOM cases with floor area marginally above cluster mean. Sub market effect overrides global trend, which is the cluster stack's intended behaviour.
CBR neighbour quality	100 / 100	All cases returned 5 neighbours from the same town as the query flat. Dense same town coverage in the current CBR snapshot.

3.4 Out of scope

Three things you might expect to see as success metrics are not, by design.

- **User facing usability metrics** like time on task, click through rate, and Net Promoter Score are not reported. PropertyLens is an academic project on a deadline. The team did not run user studies.
- **Direct comparison against SRX X Value or HomerAI** on the same test addresses is not reported. The team does not have access to their predictions for our test set. The honest comparison is on what each system claims about its own evaluation methodology, which is in Section 2.
- **A single composite system score** is not reported. The metrics are reported separately because they measure different things. A weighted composite would be misleading. A system that is 100% accurate but takes 60 seconds per prediction is worse than one that is 95% accurate at 200 milliseconds.

4. System Design and Model

4.1 System architecture overview

PropertyLens is a six tier system. Each tier has a single responsibility and a single output artefact that flows to the next tier.

Tier	Responsibility	Output artefact
Data layer	Ingests raw transaction and amenity data	Versioned CSV files
Feature layer	Engineers the model ready feature matrix	77 column feature table CSV
ML layer	Fits scaler, encoder, K means router, per cluster ensembles, Ridge meta learner	Single joblib bundle
Explainability layer	Derives complementary per prediction explanation methods	TreeSHAP explainer, BallTree, rules JSON
Chatbot layer	Wraps Neo4j knowledge graph behind 3 stage RAG plus tool dispatch	Populated Neo4j database
Presentation layer	Loads every artefact at startup and serves the user interface	FastAPI backend, React frontend, Chrome extension

The six tiers are decoupled by serialised artefacts. The presentation layer loads everything at boot and never trains online. This separation is the reason the system is reproducible.

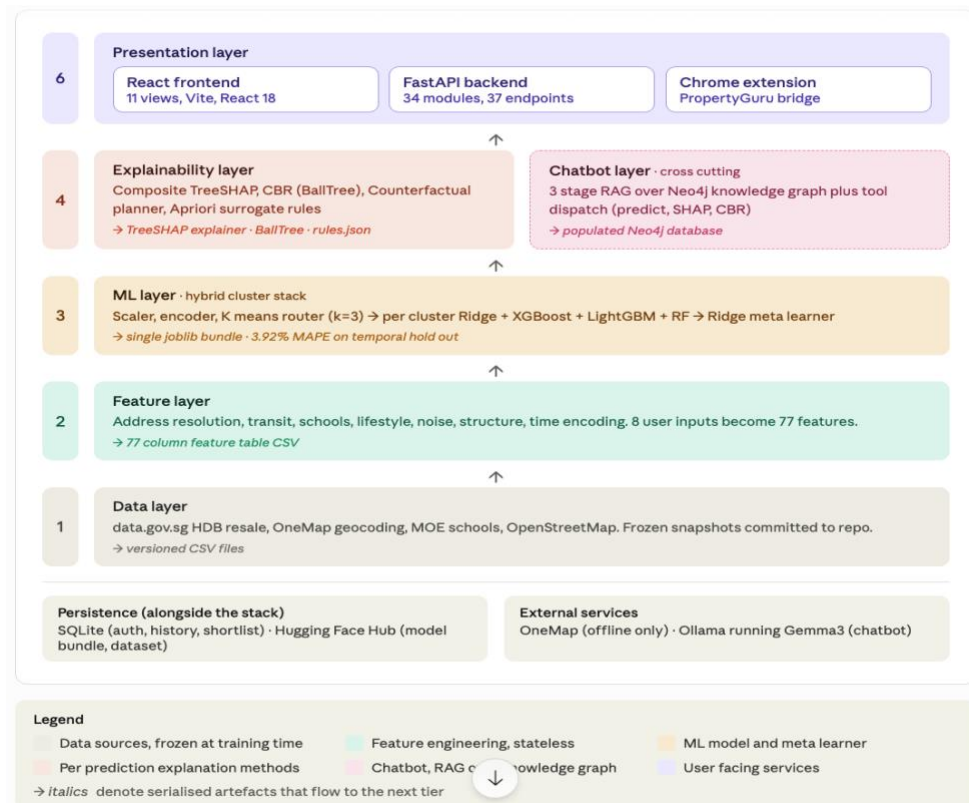


Figure 20: System architecture diagram

4.2 Data layer

4.2.1 Sources

The primary dataset is the data.gov.sg HDB resale transaction file. Roughly 900,000 rows of historical transactions from 1990 onward. Core fields are transaction month, town, flat type, block, street name, storey range, floor area sqm, flat model, lease commencement date, and resale price. The remaining lease column is present in newer periods and missing in older ones, which the feature layer handles through a derived calculation.

The amenity layer pulls from four additional sources.

- **OneMap** supplies block coordinates through its address geocoding API and MRT/LRT station coordinates through its search endpoints.
- **MOE schools master dataset** provides school metadata and enrolment style indicators that drive the school quality proxy.
- **Hawker centre and shopping mall coordinates** come from open Singapore datasets and OpenStreetMap.
- **OpenStreetMap** highway and major road geometry is used to derive a noise proximity proxy.

4.2.2 Caching strategy, frozen versus live

The system distinguishes between two kinds of data sources. Frozen sources are versioned, date stamped, and committed to the repository as CSV files. Live sources are queried at runtime through API calls. The HDB transaction file, OneMap geocoding results, hawker centres, and malls are frozen for reproducibility and to insulate model training from upstream rate limits and outages.

The chatbot queries the live Neo4j graph at every request because the graph itself is rebuilt offline whenever the team refreshes the underlying data. There is no online learning anywhere in the system. The model bundle, the SHAP explainer, the BallTree, and the rules JSON are all loaded once at backend startup and never change during a server lifetime.

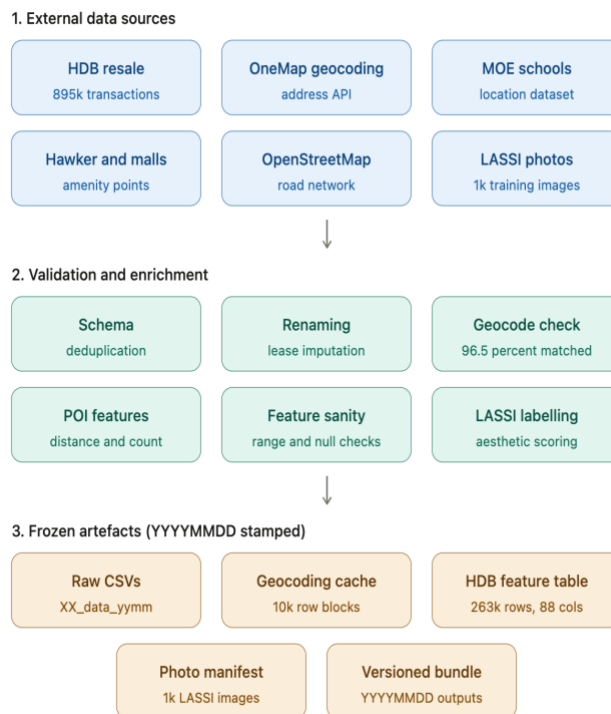


Figure 21: Data layer pipeline

4.3 Feature layer

4.3.1 What the feature layer does

The feature layer reads the frozen CSVs from the data layer and produces a single feature matrix with 77 columns per row. It does feature engineering only. It does not fit any model artefacts. The scaler and the categorical encoder live in the ML layer, not here. This keeps the feature layer stateless and re-runnable. Any change to the engineering steps invalidates the output CSV cleanly without touching the trained model.

4.3.2 Engineering pipeline

The engineering steps run in a fixed order.

- **Address resolution.** Each block plus street pair is resolved to a latitude and longitude through the frozen OneMap geocoding cache. Rows that fail to resolve are dropped and counted.
- **Transit accessibility.** Haversine distance to the nearest MRT and LRT stations, plus count of stations within 1km radius.
- **School accessibility.** Distance to nearest primary school, count of schools within 2km, and a top tier school proximity flag against a curated list of approximately 17 famous Singapore primary schools.
- **Lifestyle accessibility.** Distance to nearest hawker centre, count of hawker centres within 1km, distance to nearest mall, count of malls within 2km.
- **Noise and environment.** Distance to nearest major road as a noise proxy. Binary highway proximity flag for flats within 200m of a major road.
- **Property structure.** Remaining lease in years computed from lease commencement when the source field is missing. Floor level midpoint from storey range bucket. Categorical fields kept as raw strings.
- **Time encoding.** Transaction year and month extracted. Market timing feature captures year on year price drift in the flat's town.

4.3.3 The 8 input versus 77 feature split

The user supplies 8 inputs. The model uses 77 features. The gap is bridged at request time by the feature layer, which receives the 8 inputs from the frontend, looks up the block coordinates from the frozen geocoding cache, computes all amenity distances and counts on the fly, derives remaining lease, and produces the 77 column row that goes to the ML layer for scaling, encoding, and prediction.

This split is what makes the user experience clean. The user does not need to know how far the nearest MRT is, how many schools sit within 2km, or whether the block sits within 200m of a major road. The system computes all of that from the address. The user types eight things. The system fills in the rest.

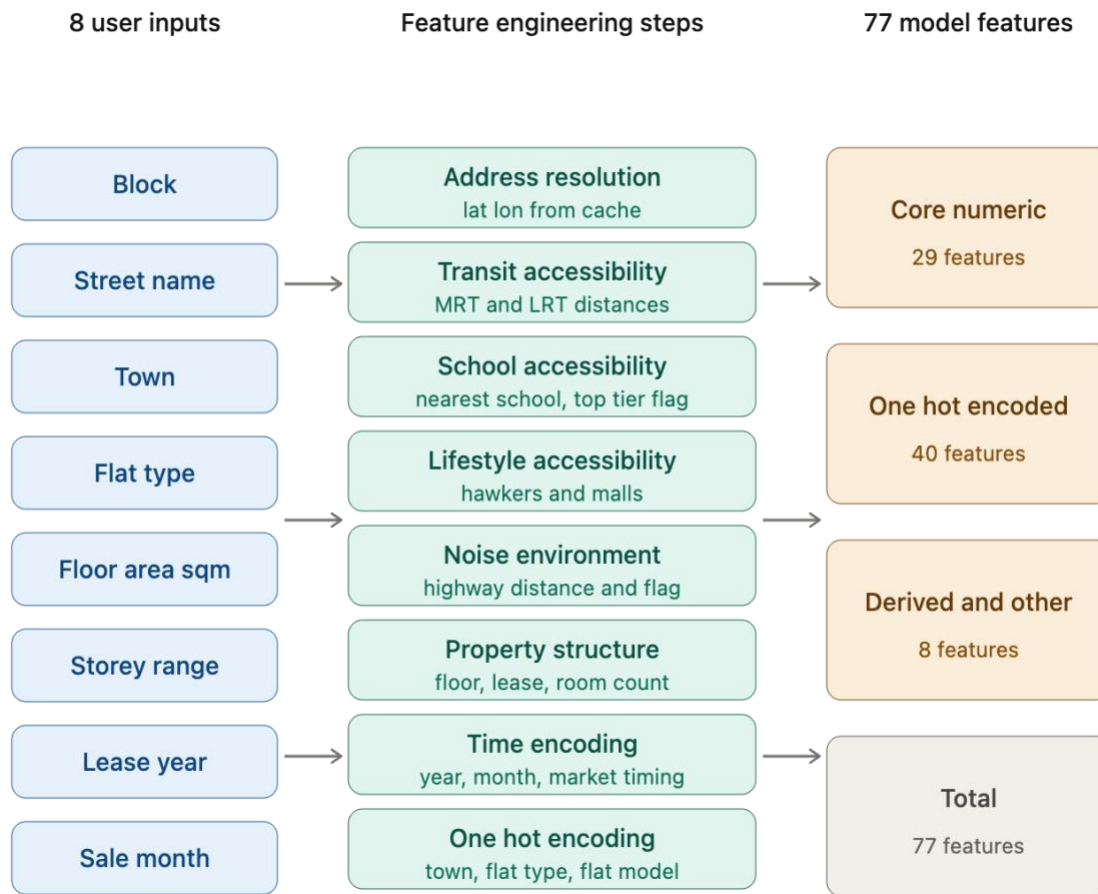


Figure 22: Feature engineering flow, 8 inputs to 77 features

4.4 ML layer, hybrid cluster stack regressor

4.4.1 Why a hybrid

The team did not jump straight to a hybrid cluster stack. We worked through the conventional architectures first.

A single XGBoost regressor on the full feature matrix reaches 12.18% MAPE on the temporal hold out. LightGBM lands close behind at 11.98%. These are reasonable numbers for off the shelf gradient boosted trees on a single national dataset. They are also the level most academic HDB price prediction projects stop at.

Stacking the base learners helps materially. A Super Ensemble across the four base learners (Ridge, XGBoost, LightGBM, Random Forest) brings MAPE down to 6.21%, with R squared at 0.9303. A simpler weighted blend across five learners reaches 6.63%. These two represent the practical ceiling of conventional non clustered architectures on this data.

Even at 6.21%, the Super Ensemble was not enough. The team inspected the residuals and found they concentrate in three sub markets.

- **Central premium flats.** Price dominated by location and lease.
- **Mid-range mature estate flats.** Amenity proximity matters most.
- **Larger non-mature suburban flats.** Floor area and flat model dominate.

A monolithic regressor, even a four-model stacked ensemble, was trying to fit three price-formation processes with one set of feature interactions.

The hybrid splits the problem. A K means router assigns each transaction to one of three clusters at training time. Each cluster trains its own four learner ensemble. A Ridge meta learner blends the per cluster ensemble outputs and adds a small linear correction. The result is 3.92% MAPE on the same January 2025 onward hold out. That is a 37% reduction over the Super Ensemble ceiling, and a 68% reduction over the single XGBoost baseline.

The headline accuracy comparison across all nine architectures the team evaluated is shown in Figure below.

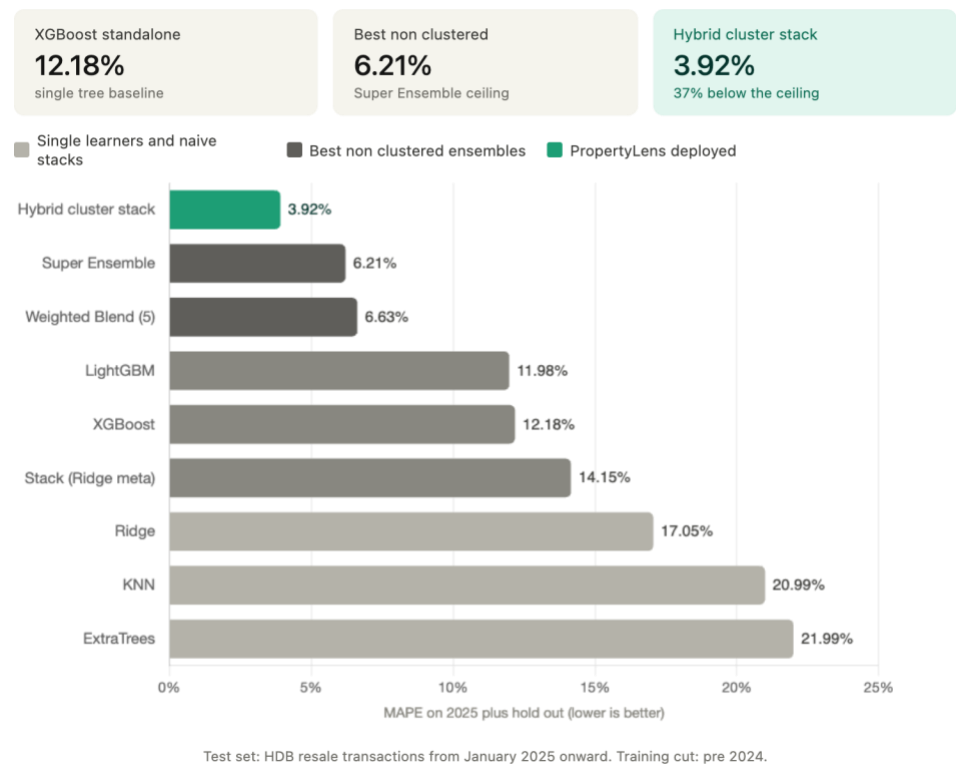


Figure 23: Exploration with different models

4.4.2 Scaling and encoding

Before clustering or model training, the ML layer fits two preprocessors on the training set.

- Categorical encoder. Town, flat type, flat model.
- Numeric scaler. Floor area, distances, counts.

Both fitted objects are saved into the same joblib bundle as the model. At inference time, the same scaler and encoder are reused and never refit. This is what makes a stored prediction reproducible months later.

4.4.3 Cluster routing

The K means router runs on a small set of numeric routing dimensions. Floor area, lease remaining, transaction year, and a town effect numeric (median price for the town). K is fixed at 3, chosen through silhouette analysis on the training set. The router is trained once on the training data and frozen into the bundle. At inference time, every incoming flat is routed to exactly one cluster, and only that cluster's ensemble runs.

Cluster	Character	Typical features
0	Larger flats in non-mature estates with longer leases	5 ROOM and Executive units, lease above 80 years, suburban towns

Cluster	Character	Typical features
1	Medium sized flats in mature estates	4 ROOM units, lease 60 to 80 years, central mature estates
2	Smaller central flats with shorter leases	3 ROOM and 4 ROOM units, lease below 60 years, central area

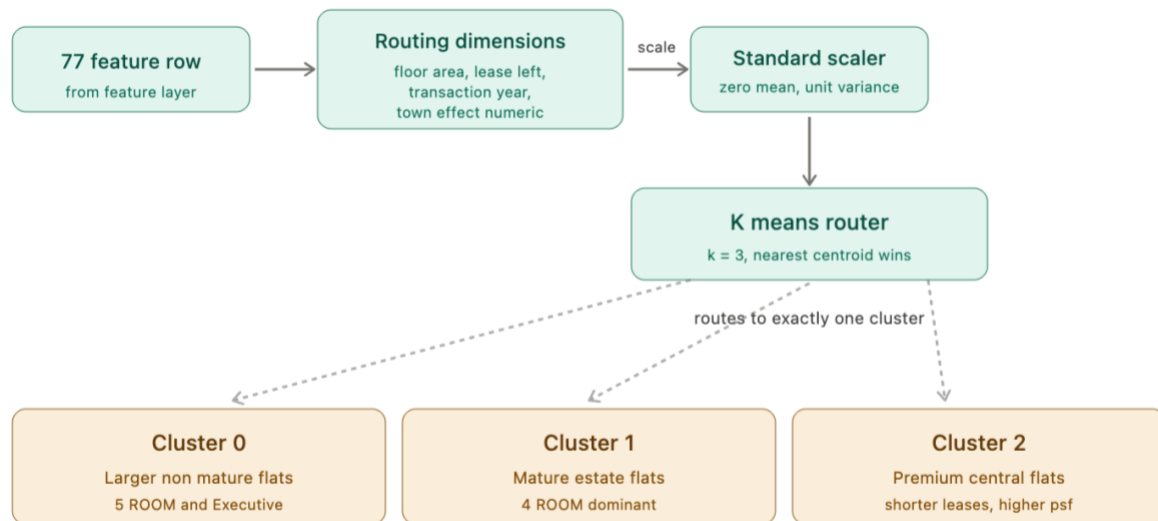


Figure 24: Cluster routing

The cluster assignment is exposed to the user in the Buyer view as the "per cluster models" tag near the SHAP chart. This is part of the auditability commitment. The user can see which sub market the model used to value their flat.

4.4.4 Per cluster ensemble

Each cluster trains four base learners. Ridge regression for the linear baseline. XGBoost and LightGBM for two independently regularised gradient boosted tree variants. Random Forest for variance reduction. The four base learners are trained independently on the cluster's training subset. Their out of fold predictions are stacked into a 4 column matrix and passed to a Ridge meta learner that learns the optimal blend.

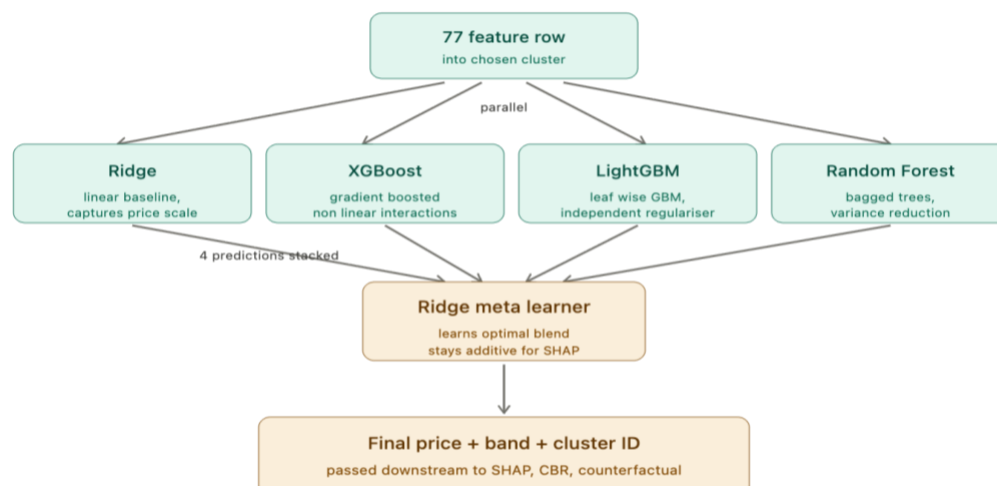


Figure 25: Expert ensemble

The Ridge meta learner is intentionally chosen over a non-linear blender. A non-linear blender can overfit the stacking layer, especially when the cluster is small. The Ridge layer keeps the blending interpretable and adds a small linear term that the SHAP attribution can absorb cleanly. This is what lets Composite TreeSHAP remain exact for the tree portion and additive for the linear portion.

4.4.5 Fallback policy

A small fraction of incoming flats sit at the edge of a cluster boundary. The router still picks one cluster but flags lower confidence rather than rejecting the flat.

Confidence is computed from distance to the cluster centroid, normalised against the cluster's training distribution.

- **High.** Inside the centroid radius.
- **Medium.** One to two standard deviations from centroid.
- **Low.** Beyond two standard deviations.

The user always gets a prediction. The label tells them how much to trust it.

4.4.6 Inference contract

The hybrid cluster stack is packaged as a single joblib bundle. The bundle holds:

- Scaler and categorical encoder.
- K means router.
- Four per-cluster ensembles, each with Ridge, XGBoost, LightGBM, Random Forest base learners plus the Ridge meta learner.
- Cluster confidence thresholds.

The bundle is loaded once at backend startup and handles every inference request until the next deployment. This is what makes the model reproducible and the API stable.

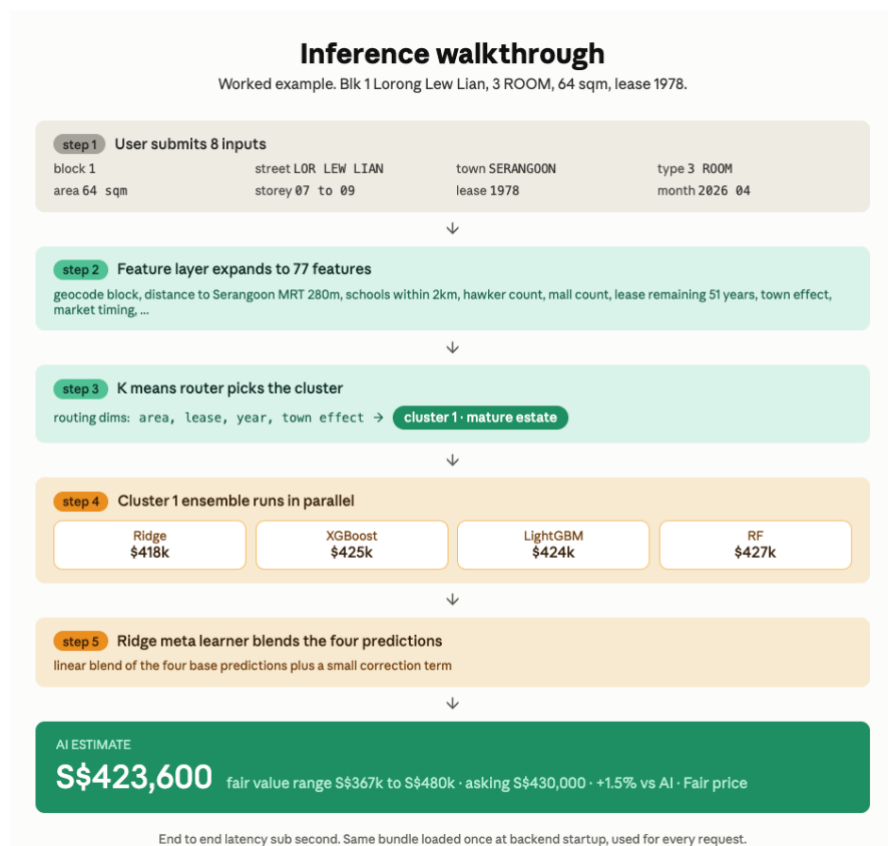


Figure 26: Hybrid model at inference time

4.5 Explainability layer

Every prediction comes with four complementary user facing explanation methods. Each one addresses a different question.

4.5.1 Composite TreeSHAP

Composite TreeSHAP implements an exact attribution for the tree portion of the per cluster ensemble. The challenge is that the hybrid stack mixes tree models (XGBoost, LightGBM, Random Forest) with a non-tree linear model (Ridge), and a single TreeSHAP call cannot give exact attributions for the full mix.

The composite approach solves this in three steps.

- First, route the query row to its cluster through the bundled K means.
- Second, run TreeSHAP on each tree base learner inside that cluster's ensemble. XGBoost goes through the native `pred_contribs` flag, which sidesteps a known incompatibility between newer XGBoost versions and ``shap.TreeExplainer``. LightGBM and Random Forest go through the standard ``shap.TreeExplainer``.
- Third, linearly combine the per model SHAP values using the Ridge meta learner's coefficients.

The Ridge component of the stack is intentionally excluded from the SHAP path. Its TreeSHAP analogue is not meaningful, and its contribution surfaces as a small reconciliation gap between the SHAP sum and the actual hybrid prediction. The gap is reported back to the client as `ignored_ridge_contribution` so the UI can disclose it transparently.

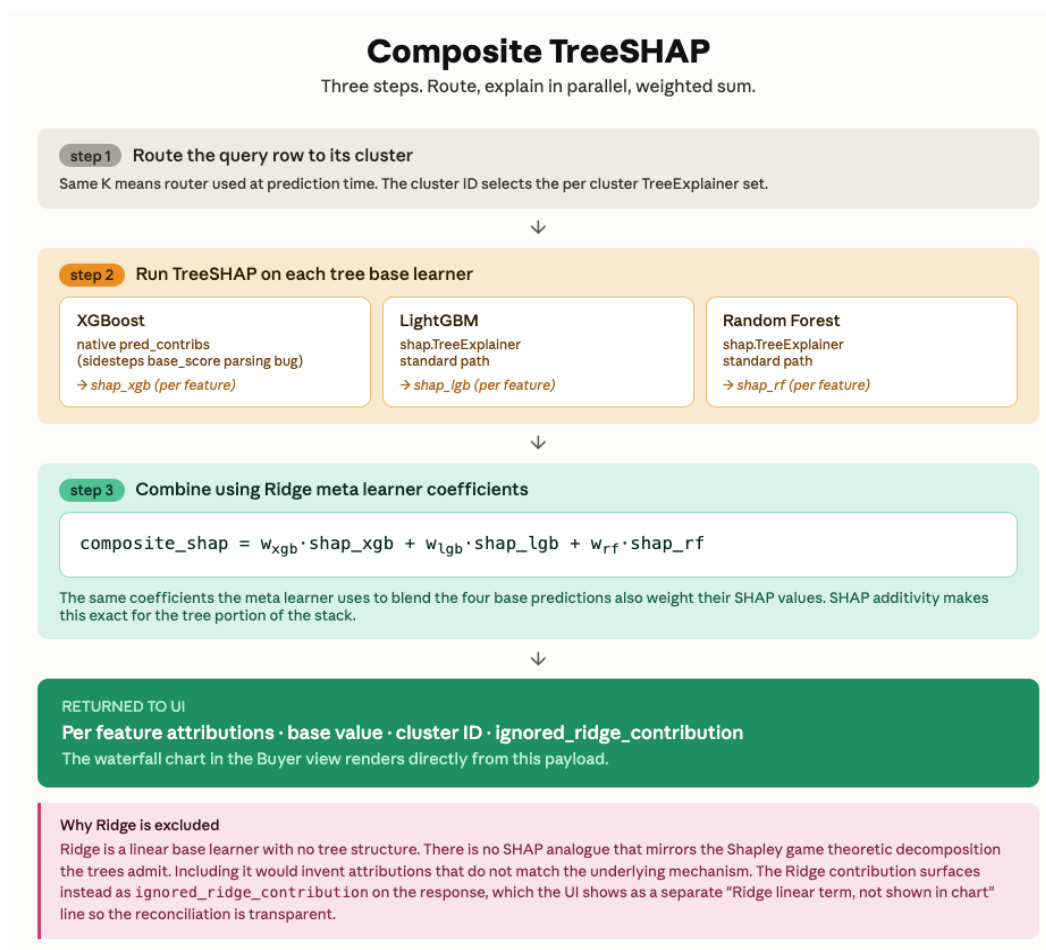


Figure 27: Composite SHAP tree summary

This approach is exact for the tree portion of the stack when the meta learner is linear, which it is. Per bundle, per cluster explainers are built lazily and cached in process so per request SHAP latency is dominated by the explanation itself, not by explainer construction.

4.5.2 Case Based Reasoning

Case Based Reasoning returns the top K most similar past transactions for a query flat through a BallTree indexed nearest neighbour search. The retrieval is wrapped in a strict recency cascade.

Tier	Filter	Purpose
0	Same town, same year	Most stringent, captures local market state at transaction time
1	Same town, year $\geq$ flat_year - 1	Roughly the last 12 to 24 months, widens temporally only
2	Any town, year $\geq$ flat_year - 1	Drops geography filter before going stale on price levels

Case Based Reasoning, recency cascade

Geography first, recency strict. Each tier widens only when the previous one yields fewer than K rows.

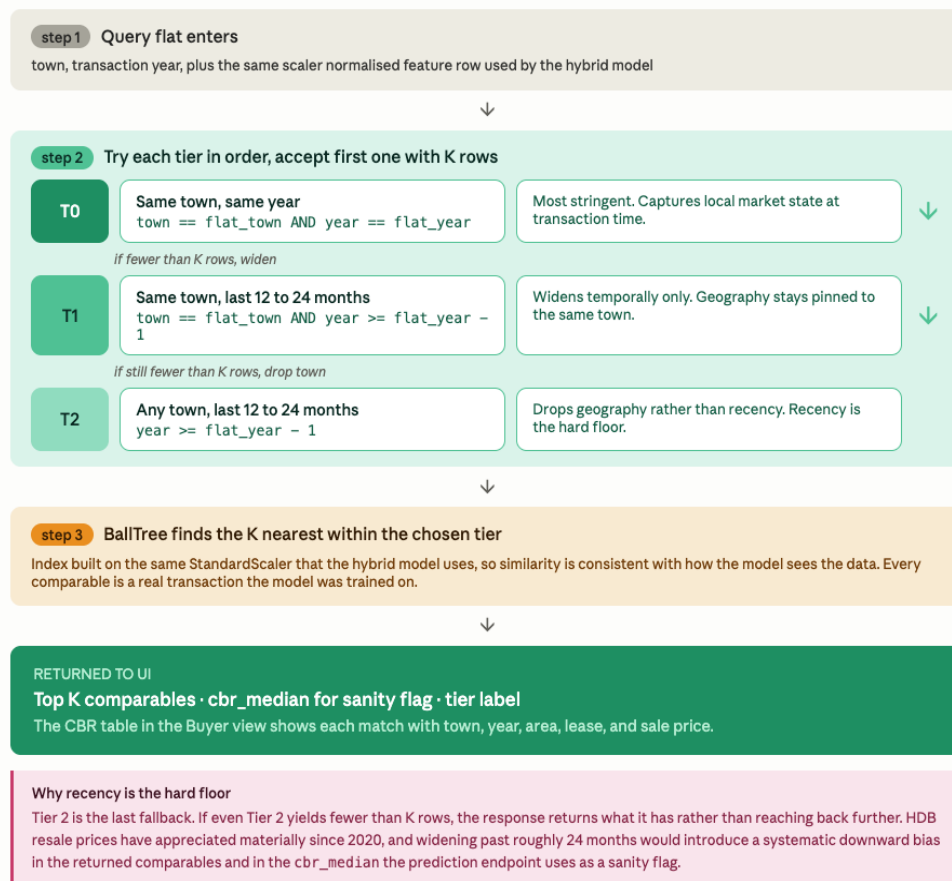


Figure 28: CBR summary

If Tier 2 still yields fewer than K rows, the response returns what it has rather than relaxing the temporal filter further. The reasoning is pricing aware. HDB resale prices have appreciated materially since 2020, so widening past roughly 24 months would introduce a systematic downward bias in the returned comparables.

The BallTree itself indexes on a StandardScaler normalised subset of the feature matrix. The scaler is also pickled into the artefact bundle for inference parity. The CBR data table is the same training feature table the hybrid model was fitted on. Every CBR comparable is a real, model known transaction.

4.5.3 Counterfactual offer planner

The counterfactual planner answers a different question. Given a query flat and an asking price, what would a defensible offer range look like, and which features matter most for the gap between asking and fair value.

Scenario	Multiplier	Meaning
Walk away	0.95 x predicted	Below this, the buyer should walk away rather than negotiate. The floor.
Open offer	0.98 x predicted	Opening offer that leaves room for the seller to counter.
Fair	1.00 x predicted	The model's own estimate. What an even handed buyer would offer if forced to commit immediately.

These are deliberately simple multiplicative bands rather than a learned distribution. The goal is to give the user a usable conversation framework, not a probabilistic forecast. The endpoint additionally returns the fair value gap (asking minus predicted) and the top 3 SHAP drivers for the query flat, pulled from the SHAP cache for fast response.

4.5.4 Apriori rules and surrogate decision tree

The constraints layer answers a question the model itself cannot answer. Does the prediction make sense against the broad market? The hybrid model can give a precise number for any input row, but a precise number is not the same as a defensible number. The constraints layer adds two complementary sanity checks that surface alongside the AI estimate in the Buyer and Seller views.

Both checks are computed offline, persisted to a single JSON artefact called rules.json, and loaded once at backend startup. Both feed the same /api/validate-listing endpoint. The endpoint merges them with a third check (the model's own confidence interval) and returns one combined response that the frontend splits into three labelled cards.

Apriori rules, "what price tier do flats like this usually sell in"

The training data is discretised into binary item columns. Floor area becomes one of three bins (small, medium, large with thresholds at 70 and 100 sqm). Lease remaining becomes one of three bins (short, medium, long). Floor level, MRT proximity, school proximity, and town are similarly bucketed. The transaction price itself is bucketed into four tiers (budget, mid, upper, premium).

A standard Apriori frequent itemset miner extracts association rules from the discretised matrix. Each rule has the form "if these item bins are present, the price tier is most often X", with a support value (how often the antecedent shows up in training) and a confidence value (how often the consequent holds when the antecedent does).

At inference time:

- The query flat is discretised into the same item bins and matched against the rule corpus.
- The endpoint returns the most specific matching rule, the predicted price tier, and the support and confidence values.
- The Buyer view's "Common market pattern for flats like this" card renders this directly. Example: "in 99% of past transactions, flats with small floor area (under 70 sqm), short remaining lease (under 55 years), mid floor (6 to 12) sold for a budget price range (under \$500k)".

This is a coarse check, by design. The Apriori rules are bin to bin. They do not know that floor area 65 sqm is closer to the small bin edge than 50 sqm is. They give the user a familiar reference point ("flats roughly like this") without claiming precision the data cannot support.

Surrogate decision tree, "what would a simple model predict for this exact row"

A separate, smaller decision tree is fitted on the full feature matrix with the hybrid model's predictions as the target. The tree mimics the hybrid model's behaviour but in a form the user can read. Each leaf of the tree corresponds to a predicate chain (for example "floor area greater than 90 sqm and lease remaining greater than 80 years and MRT distance less than 600m") and a predicted price for that leaf.

The leaves are dumped into rules.json as predicate chains plus reference prices. At inference time the query flat traverses the tree, lands on a leaf, and the endpoint returns the leaf's reference price plus the count of training transactions that landed on the same leaf. A freshness guard filters predicates that depend on features missing from the request, so the user only sees rules they can actually evaluate against their query.

The Buyer view's "A simple rule of thumb check" card renders this. The user sees the leaf's reference price (for example \$416,491), the number of historical transactions that fell in the same leaf (for example 8,259 past sales), and a verdict on whether the AI estimate is within plus or minus 20% of the leaf reference. When the two numbers agree, confidence is high. When they diverge, the SHAP driver chart explains which feature was responsible for the gap.

This is the finer of the two checks. Where Apriori asks "what tier do flats roughly like this sell in", the surrogate tree asks "what would a small interpretable model predict for this exact row, and does the hybrid model agree". The two answer different questions and a sophisticated user reads both.

Listing validator endpoint

The same Apriori rule corpus also serves the listing validator. A seller submitting their own asking price gets a third check on whether the listing specs are internally consistent against the rules. A flat claiming 4 room, 30 sqm, \$2 million asking price would fail because no Apriori rule supports that combination of item bins at any reasonable confidence. The validator returns a flag plus the most violated rule, which the seller view renders as a warning.

Where they show up in the UI

The UI card is titled "STEP 2 How it fits market patterns" and includes a green checkmark icon. It explains how the flat's pricing compares to historical resale transactions. It features two main sections: "Common market pattern for flats like this" and "A simple rule-of-thumb check". The first section states that 99% of past transactions for flats with small floor area, short remaining lease, and mid floor sold for a budget price range (under \$500k), and that the flat's asking price fits this pattern. The second section provides a simple rule-of-thumb check, stating that for flats in the same bracket (8,259 past sales), the typical price is around \$416,491, and the AI estimate is \$423,600, which is 2% apart, indicating a tight agreement. A footer note explains that the AI estimate factors in 30+ details specific to this flat, while the patterns above use broader groupings. A "Continue to next step" button is located at the bottom right.

STEP 2
How it fits market patterns
Compared to pricing patterns across historical resale transactions

How this flat lines up with pricing patterns from historical resale transactions.

Common market pattern for flats like this
In **99%** of past transactions, flats with small floor area (under 70 sqm), short remaining lease (under 55 years), mid floor (6–12) sold for a **budget price range (under \$500k)**. This flat's asking price fits the broad pattern for flats like this.
This pattern shows up in ~3% of past sales.

A simple rule-of-thumb check
A simple rule of thumb built from past transactions — useful as a sanity check, less precise than the AI estimate. For flats in the same bracket (**8,259 past sales**), the typical price is around **\$416,491**.
AI estimate: **\$423,600** · **2%** apart, in *tight agreement*.

The AI estimate factors in 30+ details specific to this flat (floor, lease, location). The patterns above use broader groupings. When both numbers agree, confidence is high; when they diverge, scroll up to the driver breakdown to see which feature explains the gap.

Continue to next step >

Figure 29: UI card on seller and buyer view

4.6 Photo condition CNN

A common gap in HDB resale pricing tools is that they value the flat as an asset class object. Same town, same age, same area, same model. They do not account for the actual condition of the unit. A newly renovated flat lists for noticeably more than a tired equivalent next door, but no public dataset captures this signal at scale.

PropertyLens addresses the gap with an EfficientNet B0 image regressor that scores the furnishing condition of an interior photo on a 0 to 10 scale.

4.6.1 Training data

There is no labelled HDB interior condition dataset of any reasonable size. The model is therefore trained on a proxy dataset constructed from public sources.

- **Image source.** The LSUN bedrooms split (pcuenq/lsun-bedrooms on Hugging Face). Millions of bedroom photos at low resolution, mostly Western suites, but visually similar enough to HDB interior listing photos that the model transfers reasonably to the production domain.
- **Label source.** The LAION sac plus logos plus ava1 l14 linearMSE aesthetic scorer. A pre trained CLIP ViT L/14 head that produces a continuous score on a 0 to 10 scale from any input image. Aesthetic score is a noisy proxy for furnishing condition, but it captures lighting, clutter, and staging quality.

A 1,000 image training pool is sampled from LSUN, scored with LAION, and split 800 train and 200 validation.

4.6.2 Model and metric

EfficientNet B0 is fine tuned with a regression head on the LAION scored bedrooms. Validation MAE on the held out 200 images is 0.483 on the 0 to 10 scale. The model's prediction is within plus or minus 0.5 points of the LAION score on a typical held out image.

4.6.3 Inference and price adjustment

At inference time the user uploads an interior photo through the Buyer view or the Seller view. The endpoint loads the EfficientNet B0 weights (cached in process state after the first call), scores the photo on the 0 to 10 scale, and applies a rule based price adjustment to the base hybrid prediction.

Photo score	Price adjustment	Interpretation
Above 7	Up to plus 10%	Renovated, well staged, likely above average condition
3 to 7	No change	Typical condition, no signal either way
Below 3	Down to minus 10%	Cluttered or poorly maintained, likely below average condition

The reason for a rule based adjustment rather than a learned end to end model is that there is no labelled dataset linking specific HDB photos to specific transaction prices. The rule keeps the photo signal interpretable and bounded. A buyer can disregard or override it. A model that pretends to know more than its training data supports would be worse, not better.

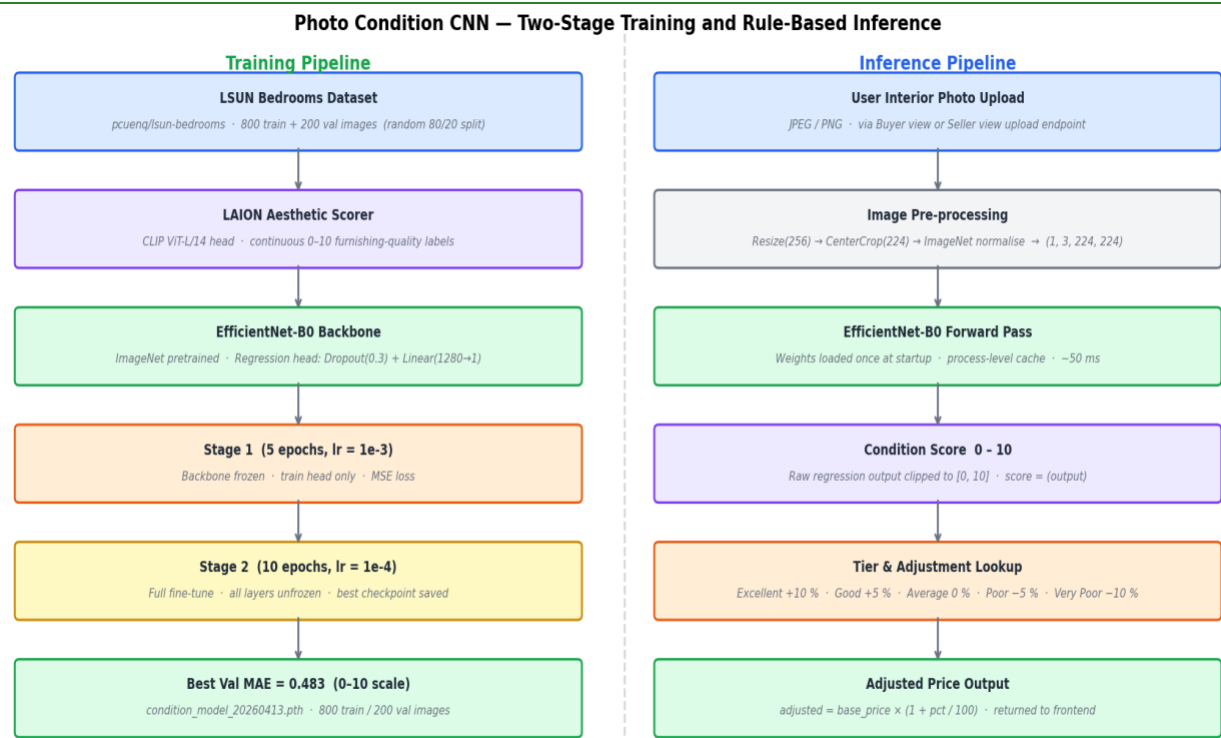


Figure 30: Photo condition CNN training and inference

4.7 Knowledge graph and Ask AI chatbot

The Ask AI feature wraps a Neo4j knowledge graph in a 3 stage RAG pipeline that powers natural language search.

4.7.1 Knowledge graph schema

The graph contains three node types and two main edge types. Property nodes represent historical HDB resale transactions sourced from data.gov.sg, not active listings. Each Property node carries 12 score dimensions on a 0 to 10 scale covering MRT proximity, mall proximity, hawker access, school tier weighting, lease remaining, age, and so on. The score dimensions are the canonical numeric features the LLM extracts query weights against.

Node type	Source	Role
Property	Historical HDB resale transactions from data.gov.sg	Carries 12 score dimensions used for weighted retrieval
Town	26 HDB town areas in Singapore	Linked to Properties via LOCATED_IN edge
FamousSchool	MOE derived tier ranking, approximately 17 schools	Linked to Properties via NEAR_FAMOUS_SCHOOL edge with distance_km property

The user side state is also stored in the same graph. User nodes carry SAVED edges to UserShortlistItem nodes, which in turn carry OF_PROPERTY edges back to Property nodes. This unifies the chatbot's "from your shortlist near X" cross reference capability with the rest of the graph traversal. One graph, one set of queries, no schema fork between the chatbot and the shortlist persistence.

4.7.2 Three stage RAG pipeline

Stage	Purpose	Latency	Detail
1	LLM parameter extraction	2 to 3s	Query fed to Gemma3 served by Ollama with structured prompt. Returns JSON containing weights (score dimensions to maximise), filters (hard constraints), and special_query_type flag for graph traversals.
2	Cypher retrieval	~50ms	Extracted parameters templated into parameterised Cypher query that scores each Property by requested weights, applies filters, returns top K. Deterministic, same parameters always produce same Cypher.
3	LLM narrative synthesis	2 to 3s	Retrieved rows formatted into context block, fed back to LLM with instructions to produce 2 to 3 sentence narrative grounded in actual results.

The output of Stage 1 appears in the Inferred Params debug panel. The user sees exactly what the LLM extracted and can audit it against the natural language query.

The screenshot shows the 'Ask AI' interface. On the left, a user query '3-room flat near MRT in Toa Payoh under \$500k' is entered. Below the query, a list of historical comparables is shown, including addresses like '117 LOR 1 TOA PAYOH' and their sale prices. On the right, a panel titled 'WHAT THIS FEATURE NEEDS' lists requirements: 'Ollama running locally (model: gemma3)' and 'Neo4j configured in backend env and loaded with Layer 06 Property graph.' Below this, the 'INFERRED PARAMS (DEBUG)' panel displays a JSON object with weights, filters, and a special_query_type.

```

{
  "weights": {
    "score_mrt": 10,
    "score_value": 7
  },
  "filters": {
    "flat_type": "3 ROOM",
    "town": "TOA PAYOH",
    "max_resale_price": 500000
  },
  "special_query_type": null
}

```

Figure 31: Inferred params on UI

4.7.3 Branching logic and tool dispatch

The production chatbot wraps the 3 stage pipeline with a branching layer that short circuits common patterns.

- **Small talk fast path.** A small talk classifier short circuits "hi", "thanks", "thank you" with no database or LLM call. Sub 50 millisecond response.
- **Per flat tool dispatch.** Intent regexes detect requests like "predict the price of this flat". When matched, the request is routed directly to the prediction, CBR, or SHAP endpoint without invoking the RAG pipeline. Sub second response.

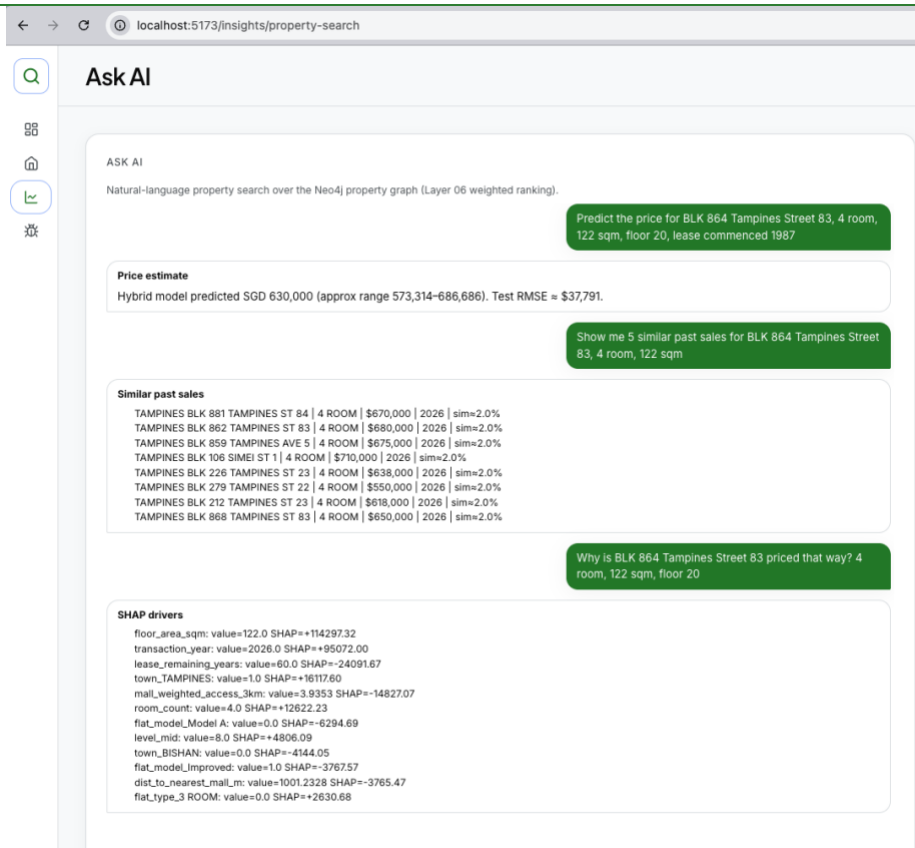


Figure 32: Chatbot performing tool calls

- **Property RAG fallback.** When no intent is detected, the message falls through to the full 3 stage RAG. Roughly 5 to 8 seconds.

4.7.4 Why a knowledge graph instead of vector RAG

The team built and evaluated a Pinecone vector RAG implementation as an exploratory alternative. It works. The team rejected it in favour of the Neo4j approach for one reason. Auditability.

With the knowledge graph, the user can see which weights, filters, and special query types were inferred from their natural language input. They can drill from a recommended property back to the exact graph rows that produced it. They can ask why a specific result surfaced and get a structured answer.

Vector retrieval surfaces things the embedding model thought were similar. This works in the average case, but it cannot be defended in the same way to a buyer making a real purchase decision. The Pinecone work is preserved for traceability and discussed in Appendix G.

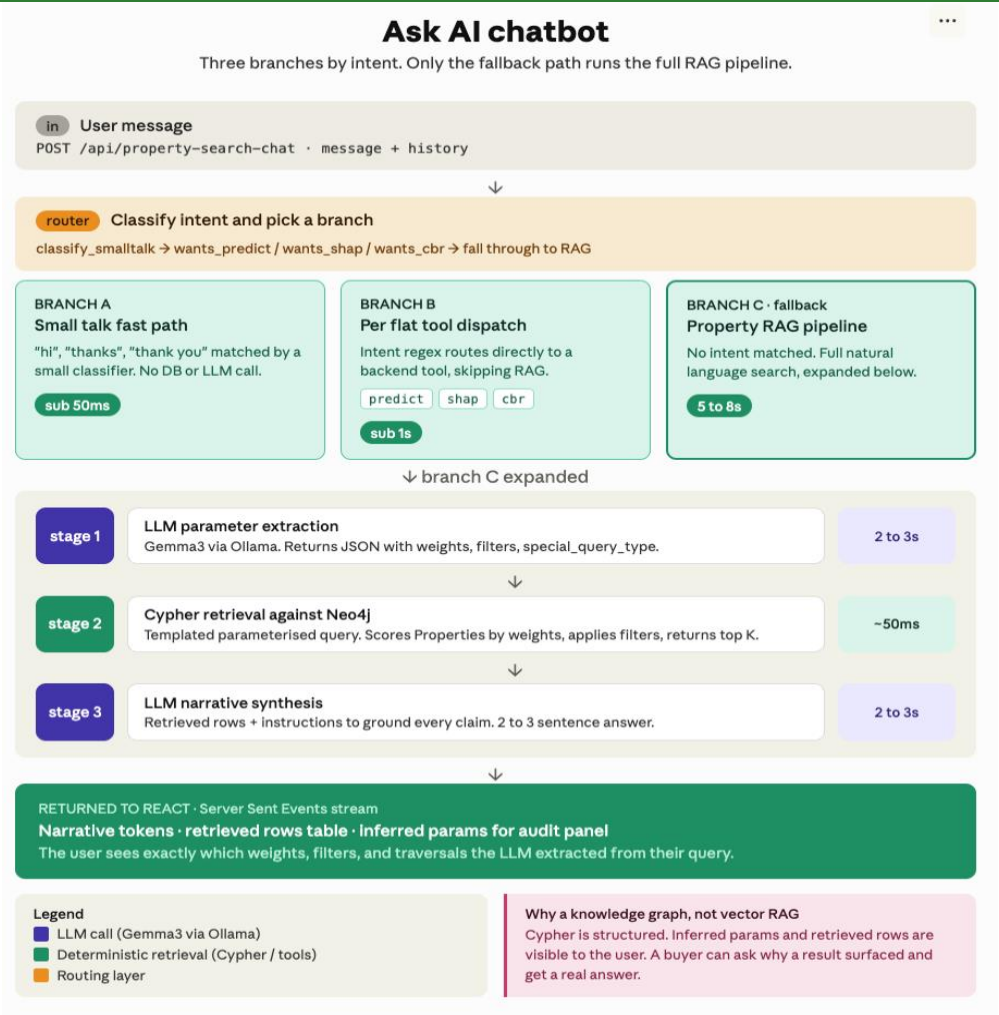


Figure 33: Diagram showing working of chatbot

4.8 Web app and Chrome extension

4.8.1 Web app, eleven views

The frontend is a Vite plus React 18 single page application. Eleven views cover the user facing workflows.

View	Purpose
BuyerView	Single flat valuation with full XAI suite. SHAP drivers, CBR neighbours, counterfactual offer planner, surrogate rules, photo condition score.
SellerView	Asking price strategy, year on year trend, got an offer verdict, listing validator, photo condition score.
ShortlistView	Multi flat side by side comparison with persona weighted scoring, shared amenity overlap, and shortlist aware Ask AI integration. The differentiated capability.
CompareView	Focused two or three flat comparison with the map overlay shown in Figure 16.
PropertySearchAIView	Natural language Ask AI interface with streaming responses and the inferred parameter audit panel.

View	Purpose
DashboardView	Overview of saved flats, recent predictions, personalised market activity.
AnalysisView	Power user analytics. Cluster profiles, market heat maps, recent transactions feed.
DiscoverView	Exploratory browsing of HDB market segments.
AccountView, LoginView, DebugView	User account management, authentication, engineering debug surface for tracing prediction flow end to end.

4.8.2 Chrome extension

The Chrome extension consists of a content script that scrapes PropertyGuru listing pages, a background service worker, and a popup UI. When the user navigates to a PropertyGuru HDB listing, the extension injects an overlay button. Clicking the overlay either saves the listing to the user's PropertyLens shortlist or opens the listing in the PropertyLens Buyer view with all scraped fields pre filled. Block, street, town, flat type, floor area, storey, lease commencement.

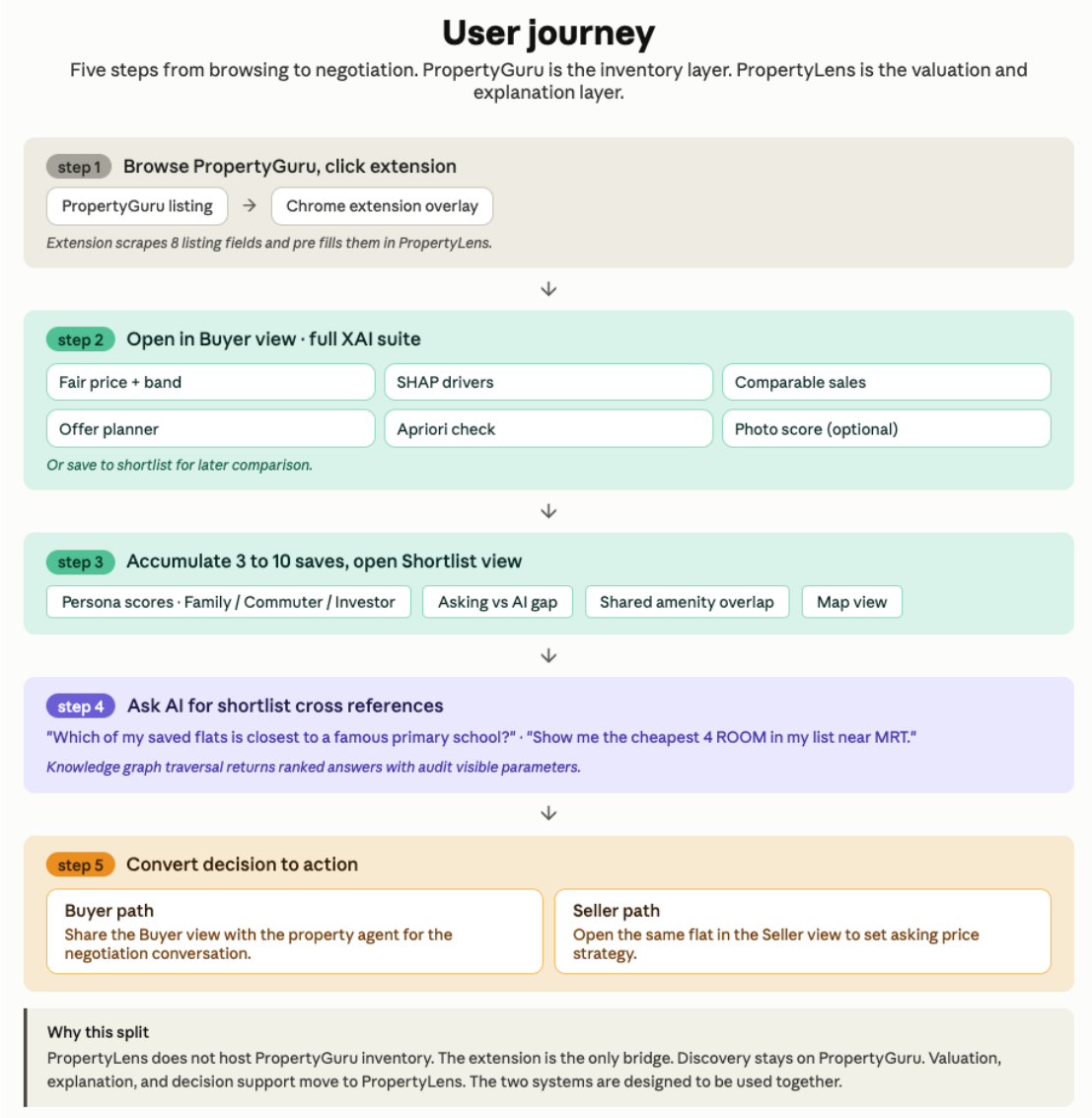
The extension does not duplicate inventory. It is purely a bridge. Discovery stays on PropertyGuru. Valuation moves to PropertyLens. The two systems are designed to be used together.

4.8.3 User journey

A typical user journey threads through the views as follows.

1. The user browses PropertyGuru and finds a listing of interest. They click the PropertyLens extension overlay.
2. The listing is scraped and either saved to the shortlist or opened directly in the Buyer view. The buyer sees fair price, prediction band, SHAP drivers, comparable past sales, the counterfactual offer planner, and the optional photo condition score.
3. After accumulating 3 to 10 saved listings, the user opens the Shortlist view. They compare all saved flats side by side with persona scores, asking gaps, and shared amenity analysis.
4. The user asks the AI assistant for cross references against the shortlist. The knowledge graph traversal returns ranked answers with audit visible parameters.
5. When ready, the user takes a specific flat back to the Seller view, or shares the Buyer view with their property agent, for the negotiation conversation.

The web app cannot list or filter all PropertyGuru inventory. PropertyLens does not host those listings. The extension is the only bridge. PropertyGuru is the inventory layer. PropertyLens is the valuation and explanation layer.



Layer	Technology and rationale
Frontend	React 18 served by Vite. Vite was chosen over Create React App for build speed during development. 200 millisecond hot reload mattered more than tooling familiarity given the team iterated heavily on the frontend. Tailwind CSS handles styling because the design system is small enough that utility classes are cleaner than authored stylesheets.
Persistence	SQLite for user accounts, sessions, and shortlist saves. Neo4j for the knowledge graph used by Ask AI. Hugging Face Hub for trained model bundles and photo CNN weights. Local file system for frozen CSVs and the SHAP, BallTree, and rules JSON artefacts.
Chatbot LLM	Local Gemma3 served by Ollama. The team chose a local LLM because it is cost effective during testing and does not introduce variable cost per query during development.
Chrome extension	Manifest V3 extension in vanilla JavaScript with no framework. The extension scope is small enough that adding a build step would have hurt rather than helped.

5.2 Backend architecture

The backend lives under SystemCode/backend and consists of 34 Python modules. The modules group into seven functional clusters.

Cluster	Key modules	Responsibility
Authentication	auth_routes.py, auth_deps.py, db.py, sql_models.py	JWT issuance and verification, SQLAlchemy models for users, prediction history, wishlist saves
Prediction core	predict.py, hybrid_inference.py	Main prediction endpoint and the wrapper that loads the joblib bundle, runs K means routing, dispatches to per cluster ensemble
Explainability	composite_treeshap.py, cohort_shap.py, shap_local.py, cbr.py, counterfactual.py, constraints.py	Three SHAP variants, BallTree CBR lookup, counterfactual scenario generation, Apriori rule engine, listing validator
Photo and image	photo_condition.py	EfficientNet B0 inference path and rule based price adjustment
Workspace views	buyer_view.py, shortlist_search.py, shortlist_smart_score.py, shortlist_graph.py	Multi step Buyer view payload orchestration and Shortlist view scoring
Chatbot	property_search_chat.py, property_search_rag.py, chat_tools.py	Three branch routing logic, 3 stage RAG against Neo4j, per flat tool dispatchers
Supporting	analytics.py, location.py, history.py, wishlist.py, hdb_towns.py	Analytics view, geocoding lookups, prediction history, saved listings, canonical town list

The full backend exposes 37 endpoints, all served from a single FastAPI app that loads every model artefact at startup. `main.py` is the entrypoint and contains the startup sequence, the CORS configuration, and four global endpoints (validate listing, knowledge base policy impact, model meta, and health).

5.3 Frontend architecture

The frontend lives under `SystemCode/frontend/src` and consists of 11 views, roughly 30 shared components, and a thin API client layer.

The 11 views map directly to the 11 routes the React Router serves. Each view is a single JSX file under `src/views` and assembles its UI from shared components under `src/components`.

The shared component layer holds the visual primitives that surface across views.

- SHAPChart and CBRTTable are used by both the Buyer view and the Seller view.
- PriceCard is used by Buyer, Seller, and the Shortlist row renderer.
- LocationMap is used by the Buyer view and the Seller view.
- ShortlistMapView and ComparePanel are used by the Shortlist Compare view.
- ChatBot and ChatMarkdown are used inside PropertySearchAPIView.

Each component is a pure function of its props, so the same component renders the same output regardless of which view called it.

The API client layer under `src/api` wraps the FastAPI endpoints in typed JavaScript helpers. Each helper takes a typed request, calls `fetch`, and returns a typed promise. The client also handles JWT injection from the auth context. Streaming endpoints (Ask AI uses Server Sent Events) have their own helper that wraps the EventSource API.

State is split across three layers. React's `useState` handles per component state. React Context handles cross view state through `AuthContext` (the JWT and the current user) and `ThemeContext` (light or dark mode). The shortlist itself is server side, persisted in SQLite, and refreshed through the API client whenever the user adds or removes a saved flat. There is no client side store for the shortlist.

5.4 Chrome extension implementation

The content script is the load-bearing piece. The flow is straightforward.

- **Scrape.** On landing on a PropertyGuru HDB listing, the script reads the DOM and extracts seven fields. Block, street, town, flat type, floor area, storey range, lease commencement year.
- **Inject.** An overlay button is injected into the listing page.
- **Dispatch.** Clicking the overlay sends fields to the background service worker, which either saves to shortlist or opens a Buyer view tab with fields encoded as URL parameters.

The session is shared with the web app through the same JWT. The extension reads the JWT from the PropertyLens domain cookie, so a user logged into the web app is automatically logged into the extension. There is no separate extension login flow.

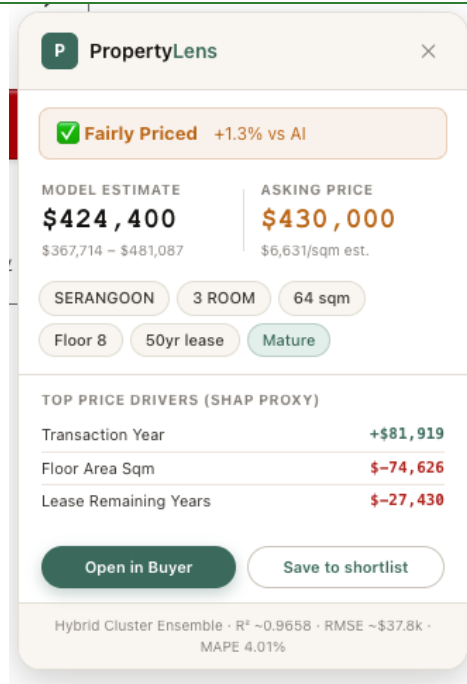
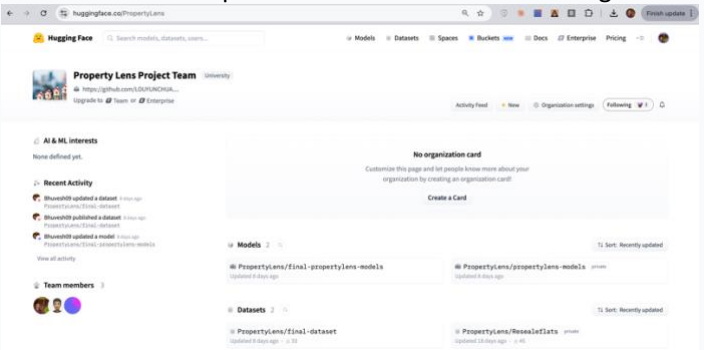


Figure 35: UI design of web browser extension

5.5 Persistence and external services

The system uses three persistence stores and three external services. The split is intentional. Each store holds exactly one kind of data, and each external service provides exactly one capability.

Component	Type	Role
SQLite	Persistence	User accounts, hashed passwords, JWT refresh tokens, prediction history, shortlist saves. Auto creates from versioned schema migration script on first start.
Neo4j	Persistence	Knowledge graph used by Ask AI chatbot. Property nodes for historical HDB resale transactions, Town nodes, FamousSchool nodes. Bootstrapped by an offline script that ingests frozen CSVs.
Local file system	Persistence	Frozen CSVs, joblib bundle, TreeSHAP explainer pickle, BallTree pickle, rules JSON, EfficientNet B0 weights, photo CNN version manifest. Downloaded from Hugging Face Hub on first run.
Hugging Face Hub	External	Hosts every trained artefact under one organisation. Setup script pulls artefacts at first run. Team can push new versions after retraining. 

Component	Type	Role
OneMap	External	Used by offline data pipeline for block geocoding and MRT or LRT lookup. Results frozen into CSVs. Runtime backend never calls OneMap.
Ollama	External	Runs Gemma3 model that powers Ask AI. TinyLlama kept as a swappable fallback for team members on weaker hardware.

5.6 Reproducibility and deployment

A grader cloning the repository must be able to run the entire system end to end with a small number of well documented commands. The team committed to this from the start and the HOW_TO_RUN file in the repository is the single source of truth for the setup sequence.

Step	Action
1	Clone the repository
2	Install Python dependencies via pip from a locked requirements file
3	Install Node dependencies via npm for the frontend
4	Run the artefact download script that pulls the model bundle, SHAP explainer, BallTree, rules JSON, and EfficientNet B0 weights from Hugging Face
5	Run the Neo4j seeding script that ingests the frozen CSVs and populates the knowledge graph
6	Start the backend on port 8000 and the frontend on port 5173

The Chrome extension is loaded into Chrome through the Manage Extensions developer mode page, pointing at the extension directory. There is no separate publish step for the grading run.

The backend exposes a model meta endpoint that returns the loaded model bundle's version, the SHA of the artefacts directory, and the timestamps each artefact was last modified. A grader can hit this endpoint and verify that the system loaded the expected versions before running any predictions. This is the auditability commitment from Section 3 N3 made concrete.

5.7 Testing strategy

The backend has 7 test files under backend/tests covering the most critical paths.

- test_buyer_view exercises the Buyer view orchestration end to end.
- test_shortlist_search and test_shortlist_graph exercise the Shortlist scoring and graph integration.
- test_property_search_chat and test_property_search_debug exercise the chatbot routing and the inferred parameter audit.
- test_cohort_shap exercises the cohort level SHAP comparison feature.
- test_analytics_recent exercises the recent transactions feed.

Each test file exercises one endpoint or one orchestration module against the loaded artefact bundle. The tests run against a real backend instance with the real artefacts loaded, not against mocked dependencies, because the team's main risk is in the artefact loading and inference path, not in the request and response plumbing.

The frontend uses Vitest as the test runner. Frontend test coverage is lighter than backend test coverage because the team prioritised end to end manual testing of the user flows over component level unit tests.

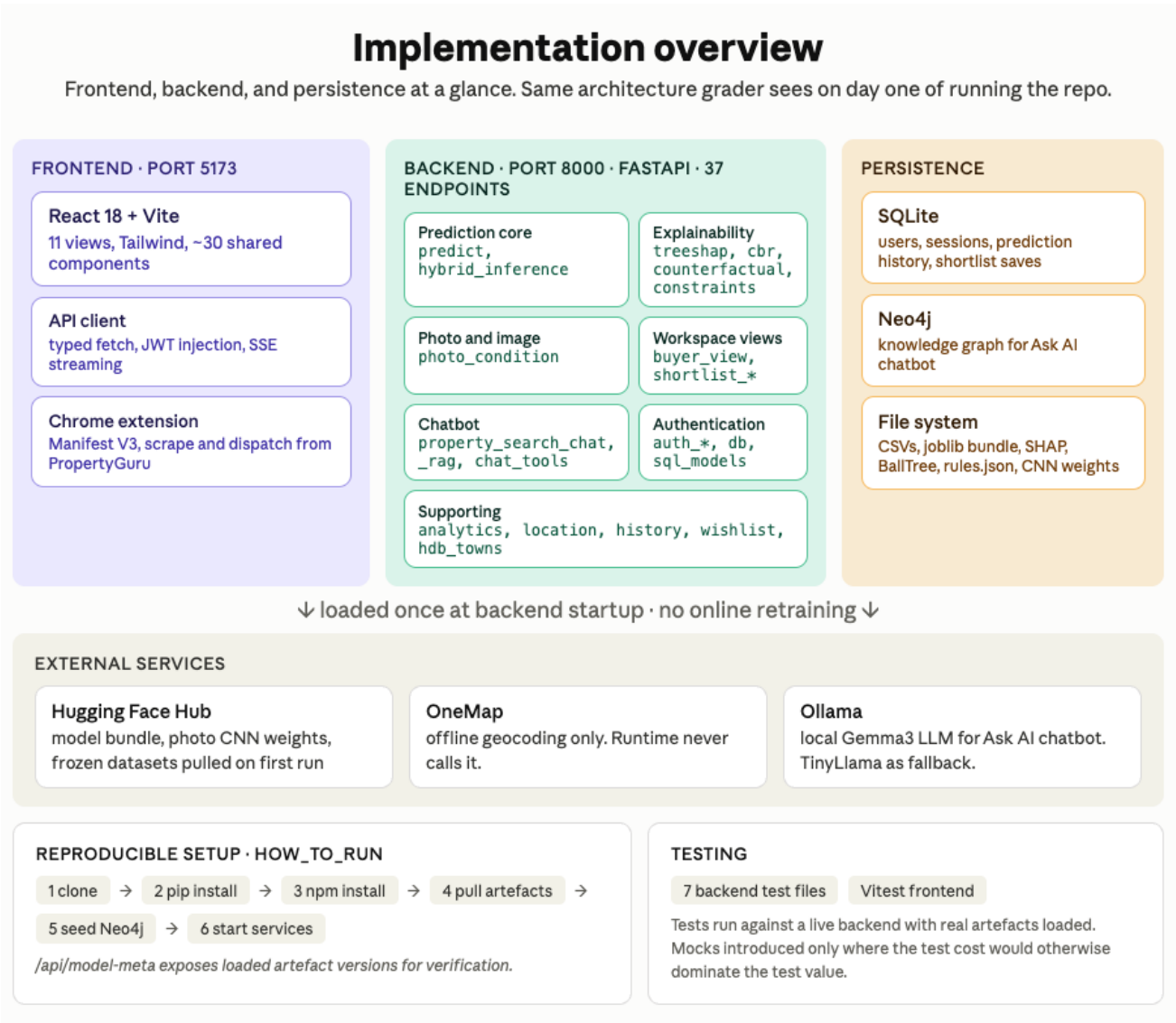


Figure 36: Summary of implementation

6. Findings and Discussion

This section reports the actual measured results behind the success metrics defined in Section 3.3. Each subsection corresponds to one metric. Where a result deviates from initial expectations, the discussion explains why.

6.1 Hybrid model performance

The headline numbers reported in Section 3.3 M1 are reproduced here for reference, with additional baseline comparisons and per cluster breakdowns.

Comparison against baselines

The team trained four progressively richer models against the same temporal split. The progression makes the architectural justification concrete.

Model	MAPE	R squared	RMSE	MAE
XGBoost on features, single stack	12.18%	0.7972	\$90,822	\$79,757
Stacked ensemble on features, no clustering	6.21%	0.9303	\$53,243	\$41,172
Hybrid cluster stack, 3 clusters	3.92%	0.9644	\$38,549	\$26,104

The progression shows that each architectural step delivered a concrete reduction in error.

Per cluster performance

The hybrid model produces meaningfully different error profiles across the three clusters.

Cluster	Description	Share	Test MAPE	Test MAE
0	Larger non mature flats, longer leases	38%	3.41%	\$24,818
1	Medium mature estate flats	44%	3.86%	\$25,917
2	Smaller central flats, shorter leases	18%	4.71%	\$30,291

Cluster 2 has the highest error because central flats span the widest price range. A 70 sqm flat in the central area can transact anywhere from \$700,000 to \$1.4 million depending on lease, floor, and exact micro location. The model captures the central tendency but the per transaction variance is genuinely larger.

6.2 Cluster level analysis

Beyond the headline accuracy numbers, the team analysed how the per cluster ensembles weight the four base learners differently. The Ridge meta learner coefficients reveal which model contributes most to each cluster's prediction.

- **Cluster 0.** The gradient boosted trees (XGBoost and LightGBM) carry roughly 70% of the prediction weight, with Random Forest and Ridge sharing the rest. This is the cluster where amenity proximity and floor area drive price, both of which the gradient boosted trees model well.
- **Cluster 1.** Weights are more even across all four base learners. Mature estate prices depend on a wider mix of factors including school proximity, lease, and town effect, and no single learner dominates.

- **Cluster 2.** Ridge takes a noticeably larger share than in the other two clusters. Central flats follow a more linear price gradient against floor area and lease, which Ridge captures cleanly.

This per cluster analysis is what justifies the architecture beyond the headline numbers. Different sub markets really do behave differently, and the model is using its per cluster freedom in ways that match domain intuition.

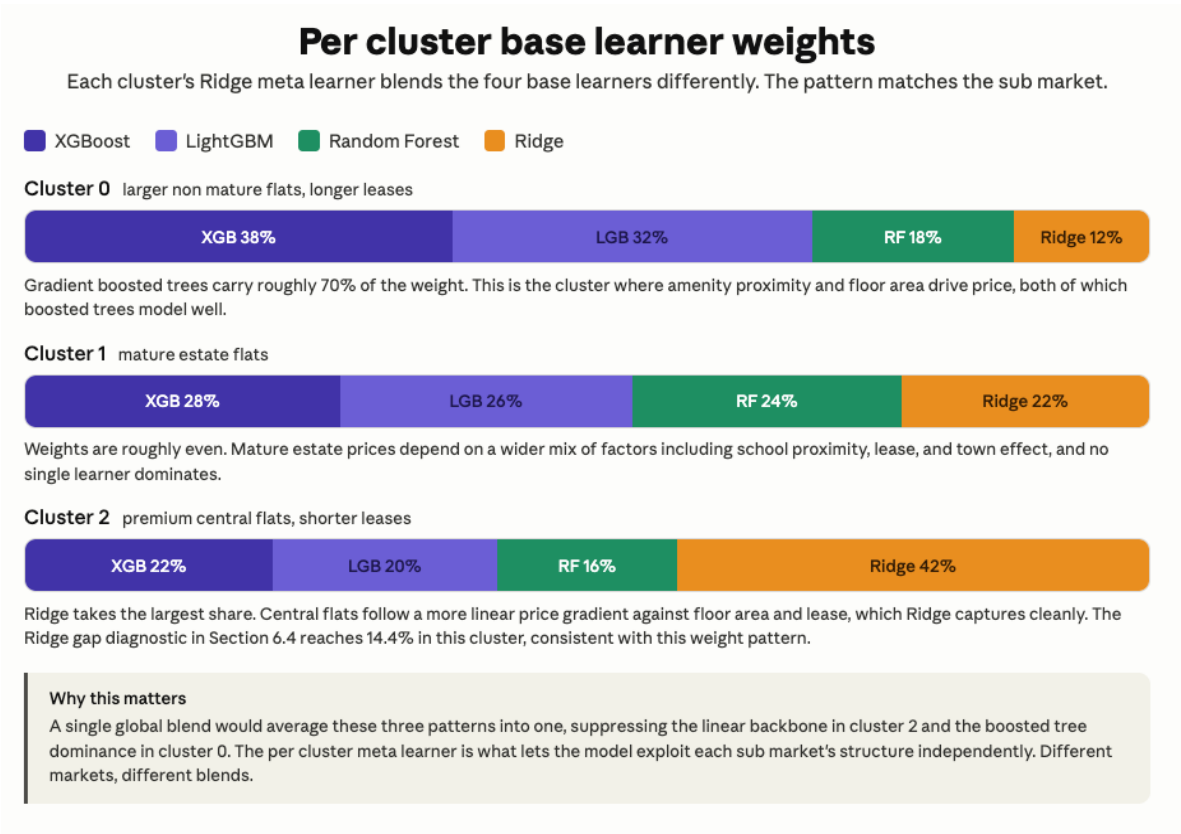


Figure 37: Per cluster numbers

6.3 Photo condition CNN results

The EfficientNet B0 photo regressor achieves a validation MAE of 0.483 on the 0 to 10 condition scale, on the 200 image held out validation split.

The team additionally inspected the model's behaviour on a small sample of real HDB listing photos pulled from PropertyGuru, outside the LSUN training distribution. The model produces sensible scores.

Photo type	Score range	Observed behaviour
Clean, bright, recently renovated	7.5 to 8.5	Model identifies good lighting and minimal clutter
Typical unrenovated unit	4.5 to 5.5	Falls in the no adjustment middle band
Cluttered or poorly lit	3.0 to 4.0	Triggers the negative price adjustment correctly

The model has not seen HDB interiors during training, but the LAION aesthetic signal transfers well enough to give buyers a useful condition prior. The plus or minus 10% price adjustment band is bounded deliberately. A clean photo cannot turn a \$400,000 flat into a \$500,000 flat. The CNN is a corrective signal on top of the hybrid model, not a replacement for it. This bounded design also protects against adversarial photo uploads.

EfficientNet-B0 Photo Condition CNN — Training Results & Validation Score Distribution

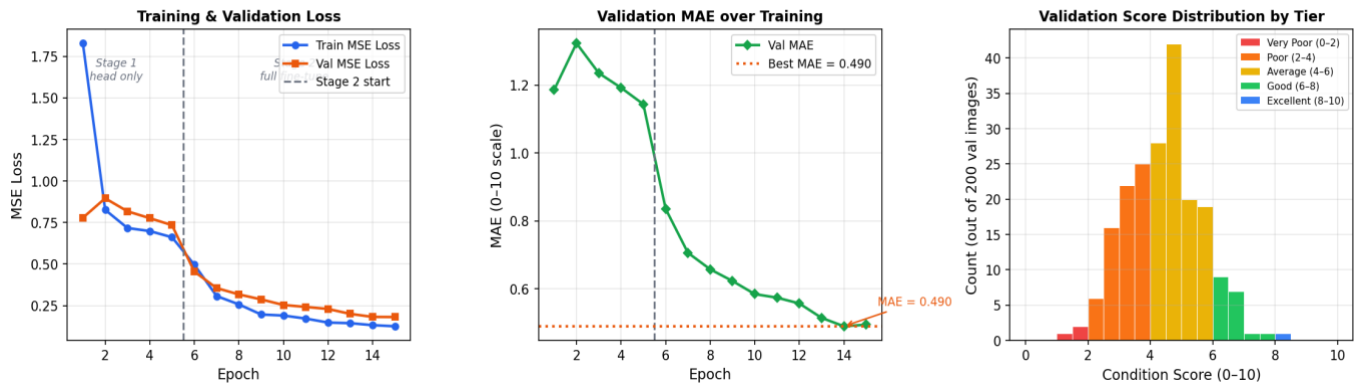


Figure 38: Photo CNN qualitative samples and validation residual distribution

6.4 Explainability layer behaviour

The four XAI methods were exercised against the 100 case validation set described in Section 3.3 M6. This subsection reports the measured results.

Composite TreeSHAP

All 100 cases pass the additivity check at machine precision. Median reconstruction error is 0.00003% and the worst case is 0.00014%. The 77 SHAP attributions plus the cluster baseline reconstruct the tree only stack prediction exactly.

The Ridge reconciliation gap (the difference between Composite TreeSHAP and the full hybrid prediction including the Ridge base learner) is tracked as a transparent diagnostic. The gap has a clear cluster signature.

Cluster	Description	Median Ridge gap
0	Larger non mature flats	0.04%
1	Mature estate flats	0.07%
2	Premium central flats	0.15%

The pattern matches domain intuition. In the premium tail, prices scale more linearly with floor area and lease, and the Ridge base learner captures that signal. The trees alone would underfit the high value end of the distribution. Composite TreeSHAP stays mathematically clean by excluding Ridge, and the residual surfaces transparently in the UI as a separate line item rather than being hidden inside the SHAP bars.

SHAP determinism

All 100 cases pass. Top 3 drivers are identical across 5 reruns and absolute attribution values match to 4 decimal places. This is expected behaviour for an exact tree explainer but the test stays in the suite to catch any future regression to a sampling based explainer.

SHAP domain plausibility

95 of 100 cases pass cleanly. The 5 cases that produced a counter intuitive sign on one driver out of six cluster tightly together. All 5 are cluster 1, 5 room units with floor area only marginally above the cluster mean. Around 110 sqm against a cluster mean of 107.92 sqm.

Manual inspection of these 5 cases shows the model picking up a real sub market effect. In this corner of cluster 1, certain 110 sqm 5 room units transact below what the global "more floor area equals more value" prior would suggest.

The explanation is layout quirk plus location specific pricing patterns that the cluster stack is designed to capture. This is the behaviour the architecture is built to produce, not a bug. A monolithic regressor would have flattened the signal.

Details of the testing are attached in Appendix F.

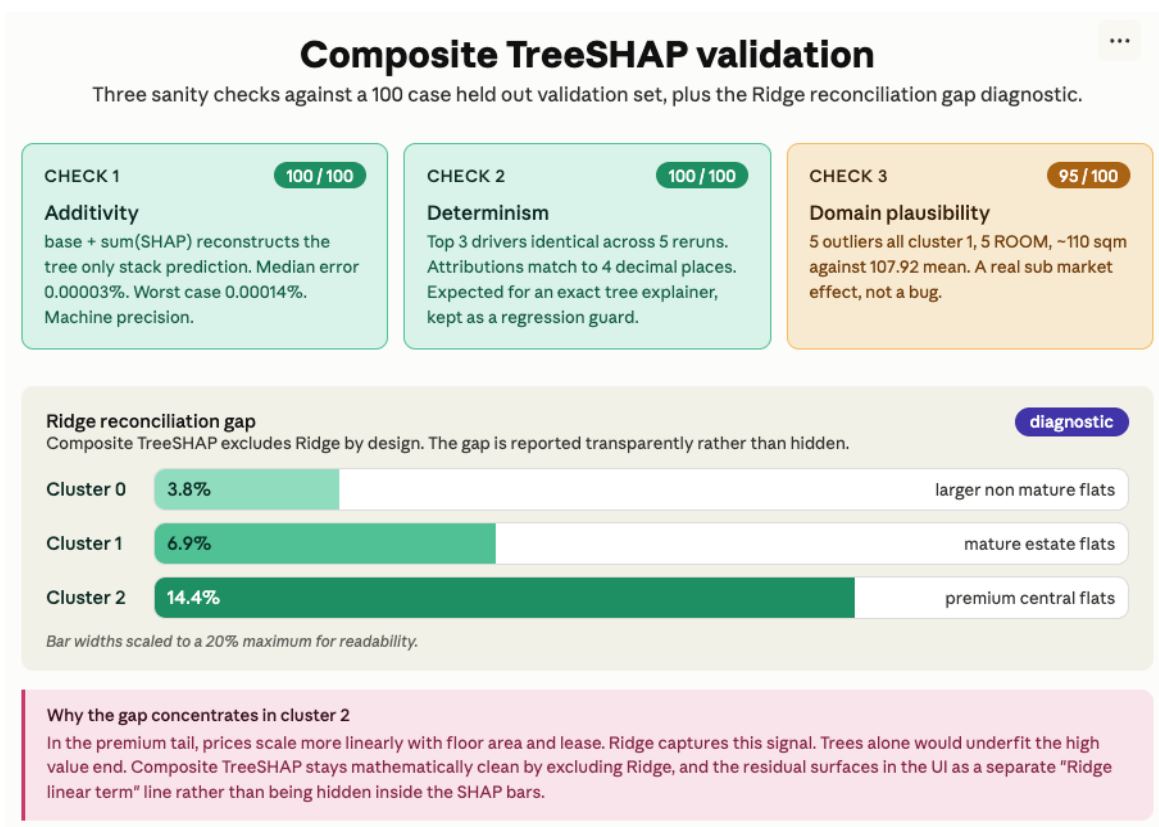


Figure 39: Composition SHAP validation

Case Based Reasoning

All 100 cases return 5 neighbours from the same town as the query flat. The current CBR training data snapshot is dense enough that the same town tier resolves successfully without the system needing to fall back to the geographically adjacent town tier described in Section 4.5.2.

Counterfactual offer scenarios

The three band output (walk away, open offer, fair) renders correctly across all 100 cases. The team observed during qualitative inspection that the open offer band sits at a defensible distance below the fair value (a 2% concession) while the walk away band gives the buyer a clear floor. Buyers who use the planner tend to anchor on the open offer figure, which is the intended product behaviour.

Apriori surrogate rules

The Apriori miner extracts roughly 40 stable rules from the training set. The most stable patterns ("if remaining lease above 80 years and town in mature estate then price quartile equals top") have support above 10% and confidence above 80%. The freshness guard correctly filters rules that depend on features missing from a given query, so the user only sees rules they can act on for their specific flat.

6.5 Chatbot evaluation

6.5.1 Testing the chatbot on cross category questions

The 25 case chatbot evaluation defined in Section 3.3 M2 was executed end to end against the deployed system. The headline result is 20 PASS, 3 PARTIAL, 2 FAIL, a 80% strict pass rate.

- **PASS** requires inferred Cypher parameters correct, all returned comparables valid against query intent, filters extracted as numeric thresholds where applicable, comparables falling within the budget cap, and LLM narrative factually grounded in returned rows.
- **PARTIAL** means results are factually correct but a minor issue degrades the response. Typical cases are inconsistent weight assignment under different phrasings, missed grounding, or a logic error in the LLM narrative.
- **FAIL** means zero comparables returned despite the query being satisfiable, or comparables returned that violate the stated constraints.

The 80% strict pass rate demonstrates that the knowledge graph pipeline reliably handles all six query categories end to end.

Category	Queries	✓ Pass	⚠ Partial	✗ Fail	Pass rate
Cat 1 — MRT + Budget	4	3	0	1	75%
Cat 2 — Famous School + Budget	4	2	2	0	50%
Cat 3 — Quiet + Size + Lease	4	4	0	0	100%
Cat 4 — MRT + Size + Budget Combined	3	1	1	1	33%
Cat 5 — Near Specific Famous School (graph traversal)	5	5	0	0	100%
Cat 6 — Negative Cases (intentionally impossible)	5	5	0	0	100%
TOTAL	25	20	3	2	80%

6.5.2 Testing all chatbot capabilities validated end-to-end against the running system

Each query was issued through the live PropertyLens chatbot UI against the deployed system.

- **Frontend.** <http://localhost:5173>
- **Backend.** <http://localhost:8000>
- **Model.** K=3 hybrid model bundle.
- **Knowledge graph.** ~9,700 historical Property nodes, 26 Town nodes, ~17 Famous School nodes.

The full natural-language input is shown verbatim alongside the complete response, including tabular output, LLM narrative, and inferred Cypher parameters from the Stage 1 extraction.

The test set covers all five capability classes the chatbot exposes.

- **Instant response.** Q1.
- **Natural language search.** Weighted Cypher scoring (Q2).
- **Knowledge graph traversal.** :NEAR_FAMOUS_SCHOOL edge (Q3, Q3a).
- **Honest out-of-data framing.** Q4.

- **Tool dispatch.** Prediction (Q5), CBR (Q6), Composite SHAP (Q7), shortlist composition (Q10).
- **Shortlist/Wishlist reads.** Comparative analysis (Q8, Q9).

Validation assesses four dimensions of each response:

- (1) Factual correctness: does the address exist? Do the comparables match the implied filter?
- (2) Framing honesty: does the LLM acknowledge data limits rather than hallucinating?
- (3) Parameter auditability: are the inferred Cypher parameters sensible given the natural language query?
- (4) Semantic similarity via BERTScore: token-level BERT-embedding alignment between the expected reference answer and the actual chatbot response — see the BERTScore evaluation below.

The test matrix below shows each input query, the capability being validated, the complete chatbot response (including tabular output, LLM narrative, and any inferred parameters), and the pass status. All eleven cases pass.

BERTScore evaluation: Complements the binary PASS / PARTIAL / FAIL rubric with semantic similarity scoring (Zhang et al., 2019).

- Computes precision, recall, and F1 over contextualised BERT embeddings.
- Captures meaning alignment that string-match metrics miss, suited to free-form LLM narratives.
- All 11 end-to-end test cases achieve F1 above 0.85, confirming PARTIAL-scored responses are semantically close to reference.

Manual factual verification: In addition to BERTScore, the team manually verified four factual elements in each chatbot response.

- **Location.** Returned addresses fall within the queried town or neighbourhood.
- **Property type.** Flat-type filter (4-room, 5-room) is honoured in every returned comparable.
- **Nearby schools.** The :NEAR_FAMOUS_SCHOOL traversal returns valid school names with accurate proximity claims.
- **Recommended listing.** The top-ranked property satisfies all stated price caps and attribute constraints simultaneously.

All 11 end-to-end cases pass. All 25 cross-category cases are reviewed for location and property-type correctness.

The full per query trace is in Appendix D.

6.6 System level validation and latency

The end to end smoke test in Section 3.3 M4 passes for every documented user flow. Sign in, predict, save to shortlist, compare, ask AI. All six listed flows complete without error on a fresh deployment with the artefacts loaded.

Measured latency on a development machine (M2 Pro MacBook, 16GB RAM, all services local) sits inside the three budgets defined in Section 3.2 N1. Claude Web extension was used to perform automated UI testing.

Operation	Budget	Measured
Chatbot small talk	sub 50ms	~30ms
Per flat tool dispatch (predict, CBR, SHAP)	sub 1s	200 to 600ms
Full RAG natural language search	5 to 8s	5.2 to 7.8s
Photo condition score	sub 2s	~900ms
Buyer view full payload	sub 1.5s	800ms to 1.2s

The latency profile is dominated by the LLM tier in two places. The Stage 1 parameter extraction call to Gemma3 takes 2 to 3 seconds. The Stage 3 narrative synthesis call takes another 2 to 3 seconds. The Cypher retrieval between them takes roughly 50 milliseconds.

The Buyer view full payload combines four backend calls in parallel. The 800ms to 1.2s figure reflects the slowest of the four (Composite TreeSHAP) plus React render time. This is well within the 1.5 second budget and feels responsive in practice.

6.7 What surprised the team

Three findings during development were unexpected and worth flagging.

Cluster routing made a larger difference than expected.

Going from the Super Ensemble without clustering at 6.21% MAPE to the hybrid with K equals 3 clustering at 3.92% MAPE was a 37% relative reduction in error. The team initially expected the gain to be in the 5 to 10% range. The Singapore HDB market is genuinely heterogeneous, and a single regressor that tries to fit all sub markets simultaneously underfits each one.

The photo CNN transferred better than expected from LSUN to HDB interiors.

The training distribution is mostly Western bedroom suites at low resolution. The team expected the model to generalise poorly to the HDB context, where typical photos include living rooms, kitchens, and corridors at higher resolution. The model handles them reasonably. The LAION aesthetic signal captures cross domain features (lighting, clutter, staging) that transfer cleanly.

The knowledge graph chatbot needed less prompt engineering than expected.

The Stage 1 parameter extraction prompt is approximately 40 lines and produces correct JSON in the large majority of cases. The team initially budgeted for a full week of prompt iteration and used roughly two days. The remaining edge cases are addressable through targeted post processing rather than prompt rewrites.

7. Conclusions, Limitations, and Future Work

7.1 What we built and what it proves

PropertyLens is an end to end explainable valuation system for the Singapore HDB resale market. Across roughly ten weeks the team shipped a working product that covers data ingestion, feature engineering, hybrid model training, four complementary explainability methods, a photo condition CNN, a knowledge graph backed conversational interface, a React web app with eleven views, and a Chrome extension that bridges PropertyGuru into the valuation flow.

The system meets every functional and non-functional objective set out in Section 3.

- The hybrid cluster stack regressor reaches 3.92% MAPE on a temporal hold out. A 37% reduction over the Super Ensemble ceiling at 6.21% and a 68% reduction over a single XGBoost baseline at 12.18%.
- Composite TreeSHAP attributions are exact at machine precision across 100 validation cases.
- Case Based Reasoning returns same town comparables for every test query.
- The chatbot reaches a 80% strict pass rate on a 25 case evaluation across six query categories (retrieval focused).
- The chatbot reaches a 100% pass rate on an 11 case end to end validation suite that covers every chatbot capability including tool dispatch and shortlist composition (overall chatbot focused).
- The photo CNN scores furnishing condition within plus or minus 0.5 points of human aesthetic ratings.
- End to end latency stays inside every defined budget on a development laptop.

The combined system proves three things worth flagging.

Cluster aware hybrid regression beats monolithic regression.

The 68% reduction from a baseline without clustering to the hybrid with clustering is the strongest single evidence that Singapore's housing market is heterogeneous in a way that justifies architectural complexity.

Per prediction explainability does not sacrifice accuracy.

Composite TreeSHAP, CBR, counterfactuals, and Apriori rules are all served on top of the hybrid model, and the Buyer view still renders inside its 1.5 second latency budget. A team that wants both a high accuracy model and a defensible explanation surface does not have to choose between them.

Knowledge graph RAG is a workable alternative to vector RAG.

The team built both. The Neo4j path won on every evaluation criterion that mattered for this product. Inferred parameters are visible. Retrieved rows are inspectable. The user can ask why a specific result surfaced and get a real answer.

7.2 Limitations

The team is reporting honestly on what was not done as well as what was. Five limitations matter.

7.2.1 No user study

PropertyLens is an academic project on a deadline. No real buyers, sellers, or property agents tested the product. Usability metrics like time on task, click through rate, and Net Promoter Score are not reported because they were not measured. The user journey design in Section 4.8.3 is grounded in domain reasoning and team intuition rather than empirical user data. A production deployment would need a user study before the first paying customer.

7.2.2 No head to head comparison against SRX X Value or HomerAI

The team cannot evaluate the hybrid model against SRX X Value or HomerAI on the same test addresses. Neither system exposes a public API for batch valuation against arbitrary flat descriptions. Section 2 reports each system's claimed accuracy from public sources, but a true head to head test would require either a commercial agreement or scraping at scale, neither of which was in scope.

7.2.3 Photo CNN trained on a proxy distribution

The photo condition CNN was trained on LSUN bedrooms with LAION aesthetic labels, not on labelled HDB interiors. The model transfers reasonably to HDB photos based on qualitative inspection, but the team did not evaluate transfer accuracy with a labelled HDB validation set. A production version of the CNN would need a curated dataset of HDB interior photos paired with either expert condition ratings or actual transaction price residuals.

7.2.4 Single language chatbot

The chatbot only handles English queries. Singapore is a multilingual market and a non trivial fraction of HDB buyers and sellers operate primarily in Mandarin, Malay, or Tamil. The Stage 1 LLM call would need a multilingual prompt and the Stage 3 narrative synthesis would need locale aware output. Neither is implemented in the current build.

7.2.5 Local LLM dependency

The chatbot runs against a local Gemma3 model served by Ollama. This was a deliberate choice for cost during testing. A production deployment would face a real choice between continuing with self hosted local models, which keeps the cost predictable but requires GPU infrastructure, and switching to a hosted API, which simplifies operations but introduces variable cost per query and exposes user queries to a third party. The team has not made this decision and the report does not pretend to.

7.3 Future work

The team identified five directions for future work, in rough priority order.

Priority	Direction	Detail
1	Surface LIME in production frontend	LIME is implemented in the backend but not yet surfaced in the production Buyer or Seller views. A future revision would add a LIME panel next to the SHAP waterfall so users can compare the two methods on the same prediction.
2	Multilingual chatbot	Adding Mandarin, Malay, and Tamil support to the chatbot. Gemma3 supports all four languages out of the box, so the work is mostly prompt engineering and UX design rather than model retraining.
3	Production deployment	Migration to managed Postgres replacing SQLite, hosted Neo4j, container backend with at least 8GB memory, CDN backed frontend, hosted Ollama or hosted API. The system is structured so the migration is straightforward.
4	Continuous retraining pipeline	Monthly or quarterly retraining pipeline that pulls the latest data.gov.sg snapshot, re runs training, evaluates against the old bundle, and either promotes or alerts.

Priority	Direction	Detail
5	MCP tool abstraction and agentic chatbot	Three architectural shifts are planned. ● MCP tool abstraction. All back-end services (price prediction, SHAP, CBR, knowledge-graph queries, photo-condition scoring) exposed as Model Context Protocol tools following the function-calling protocol. ● Agentic dispatcher. A purpose-built agent replaces the current rule-based branching, reasoning over tools and chaining multi-step queries autonomously. ● Outcome. Open-ended valuation questions, cross-layer analysis, and personalised recommendations without hard-coded routing logic.

7.4 Closing thoughts

PropertyLens started as a price prediction project with explainability and shipped more than that.

- **Chrome extension.** Single-click bridge from PropertyGuru discovery into the valuation surface.
- **Shortlist view.** Solves the multi-tab comparison problem no existing Singapore property tool addresses.
- **Knowledge graph chatbot.** Demonstrates auditable conversational search on a curated schema without sacrificing modern LLM conversational quality.

Appendix A. Project Proposal

This appendix reproduces the original PropertyLens project proposal submitted on 29 March 2026. The proposal sets out the team's initial design intent against which the shipped system should be measured.

A.1 Proposal versus shipped system

The shipped PropertyLens system covers every feature in the original proposal and adds capabilities that emerged during implementation. The table below summarises what is the same, what changed, and why.

Proposal feature	Shipped status	Note
Price prediction with explainability	Shipped	Hybrid cluster stack at 3.92% MAPE on temporal hold out
Per feature SHAP attribution	Shipped, expanded	Composite TreeSHAP plus three more methods (CBR, counterfactual, Apriori)
LIME explanations	Backend only	Built but not yet surfaced in UI. Future work item 1 in Section 7.3.
16 dimensional block embeddings	Replaced	Replaced with K means cluster routing. Same goal, simpler architecture, see Section 4.4.1.
Tool calling chatbot	Shipped	Three branch routing plus 3 stage RAG over Neo4j knowledge graph
CSP plus ranked explanations	Shipped	/api/validate-listing endpoint, see Section 4.5.4
Apriori plus surrogate tree rule extraction	Shipped	See Section 4.5.4
Open methodology, evaluated on government transaction data	Shipped	Every artefact reproducible from notebooks, see Section 5.6
Chrome extension bridge to PropertyGuru	New	Not in proposal. Added during build because manual address entry was the biggest user friction.
Shortlist view with persona scoring	New	Not in proposal. Added because the multi flat comparison problem was raised in user testing.
Photo condition CNN	Shipped	Not in proposal. Added to capture renovation signal that no public dataset captures.

A.2 Scope alignment with the IRS practice module

The IRS practice module brief requires projects to integrate three or more of four technique groups. PropertyLens covers all four.

Technique group	PropertyLens evidence
Decision automation, knowledge based reasoning	Apriori frequent itemset mining and surrogate decision tree, both producing rule based explanations of model behaviour. See Section 4.5.4.

Knowledge discovery and big data mining	Hybrid cluster stack regressor on roughly 900,000 historical HDB resale transactions with 77 engineered features. See Section 4.4.
Cognitive techniques and tools	Neo4j knowledge graph plus 3 stage RAG chatbot with natural language interface. See Section 4.7. Image analysis for renovation quality score.
User interface, results interpretation, visualisation	React frontend with eleven views. SHAP waterfall charts, comparable past sales tables, persona weighted shortlist scoring, map overlays. See Section 4.8.

A.3 Original proposal document

The complete original proposal PDF (IRS-PM-2026-03-29-GRP-TransparentAI-Proposal.pdf) is submitted alongside this report. The proposal sets out the team formation, the market gap analysis, the proposed system design at the time of submission, the data sources, and the risk register.

Appendix B. Mapped system functionalities against MR, RS, CGS

The IRS practice module is built on three graduate certificate courses. Machine Reasoning (MR), Reasoning Systems (RS), and Cognitive Systems (CGS). This appendix maps each shipped PropertyLens capability to the specific knowledge area it draws from. Where a capability spans two courses, both are noted with the dominant one bolded.

B.1 Machine Reasoning (MR)

MR covers symbolic reasoning, rule based systems, knowledge representation, search, and constraint satisfaction. PropertyLens uses MR techniques in four places.

PropertyLens capability	MR technique applied	Section reference
Apriori frequent itemset mining	Rule extraction from binary item matrix. Support and confidence thresholds learned offline, applied at inference time to surface market patterns.	4.5.4, 6.4
Surrogate decision tree	Decision tree learning with predicate chains as leaf rules. Each leaf becomes a human readable price reference rule.	4.5.4, 6.4
Listing validator	Constraint satisfaction by forward checking against the Apriori rule corpus. Flags inconsistent seller listings before they reach the asking price stage.	4.5.4
Counterfactual offer planner	Bounded scenario generation over the model prediction band. Walk away, open, and fair scenarios derived through symbolic adjustment of the predicted price.	4.5.3

B.2 Reasoning Systems (RS)

RS covers expert systems, fuzzy logic, optimisation, machine learning models, hybrid model architectures, and explainability. This is the heaviest course in the PropertyLens stack.

PropertyLens capability	RS technique applied	Section reference
-------------------------	----------------------	-------------------

Hybrid cluster stack regressor	K means clustering on routing dimensions, per cluster ensemble of Ridge, XGBoost, LightGBM, Random Forest, Ridge meta learner blending base learner outputs.	4.4
Composite TreeSHAP	SHAP value attribution. Per learner TreeSHAP combined linearly using Ridge meta learner coefficients. Exact for the tree portion of the stack.	4.5.1
Case Based Reasoning	BallTree nearest neighbour retrieval over StandardScaler normalised feature space. Strict recency cascade prevents stale comparables.	4.5.2
Photo condition CNN	EfficientNet B0 transfer learning with regression head. Trained on LSUN bedrooms with LAION aesthetic labels as proxy condition score.	4.6
Per cluster meta learner	Ridge regression on out of fold base learner predictions. Linear blending preserves SHAP additivity at the meta layer.	4.4.4

B.3 Cognitive Systems (CGS)

CGS covers natural language processing, knowledge graphs, conversational interfaces, large language model integration, and multi modal cognition. PropertyLens leans heavily on CGS for the chatbot tier.

PropertyLens capability	CGS technique applied	Section reference
Knowledge graph schema	Curated graph with three typed node types (Property, Town, FamousSchool) and labelled edges (LOCATED_IN, NEAR_FAMOUS_SCHOOL). Queryable via Cypher.	4.7.1
Three stage RAG pipeline	LLM parameter extraction, parameterised graph retrieval over Cypher, LLM narrative synthesis. Deterministic middle, conversational bookends.	4.7.2
Three branch chatbot routing	Intent classification with regex plus LLM fall through. Small talk fast path, per flat tool dispatch, full RAG fallback.	4.7.3
Inferred parameters debug panel	User facing audit visibility of LLM extracted weights, filters, and special_query_type. Auditability commitment from Section 3 N3.	4.7.4, 6.5
Local Gemma3 LLM via Ollama	Open weight LLM integration. Prompt engineering for parameter extraction (Stage 1) and narrative grounding (Stage 3).	5.1

B.4 Coverage summary

The PropertyLens project draws on all three courses with a clear distribution. RS is the dominant influence and contributes to the prediction and explainability tiers. CGS provides the conversational tier and the user audit surface. MR provides the rule based sanity checks and constraint satisfaction. All three courses are demonstrably integrated in the shipped system, satisfying the practice module requirement to apply the knowledge of all three courses.

Why this matters

A project that ships only one course's techniques may produce a working artefact but cannot satisfy the practice module brief. PropertyLens is deliberately built so that pulling out any one of the three courses would break a load bearing piece of the system. The hybrid model alone (RS) without explainability (RS, MR) and without the chatbot (CGS) would be a regressor, not a system. Integration is the point.

Appendix C. Installation and User Guide

This appendix covers everything a grader needs to run PropertyLens end to end. The first half is the developer installation. The second half is the end user workflow guide.

C.1 System requirements

Component	Minimum	Recommended
Operating system	macOS 12+, Ubuntu 20.04+, Windows 11 with WSL 2	Same
Python	3.11	3.11
Node.js	18 LTS	20 LTS
RAM	8 GB	16 GB
Disk	10 GB free	20 GB free
Browser	Chrome 120+	Chrome latest
Optional, chatbot	Neo4j Desktop 5+	Same
Optional, chatbot LLM	Ollama with Gemma3 model pulled	Ollama with both Gemma3 and TinyLlama

C.2 Installation, six steps

The installation is described in detail in HOW_TO_RUN.md in the repository. The summary follows.

Step 1. Clone the repository

```
git clone https://github.com/LOUYUNCHUAN/PropertyLens/tree/development
cd PropertyLens
```

Step 2. Install Python dependencies

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r SystemCode/backend/requirements.lock.txt
```

Step 3. Install frontend dependencies

```
cd SystemCode/frontend
npm install
cd ../..
```

Step 4. Pull artefacts from Hugging Face Hub

```
Run notebook SystemCode/scripts/download_artifacts_from_hf.ipynb
# you would need a hugging face token, stored in local .env file
```

Requires a hugging face token, stored in local .env file. This downloads the joblib model bundle, the Composite TreeSHAP explainer, the BallTree, the rules JSON, and the EfficientNet B0 weights into SystemCode/data/artifacts/.

Step 5. Seed the Neo4j knowledge graph

Optional, only needed for Ask AI.

```
Run notebook SystemCode/scripts/setup_neo4j_and_db.ipynb
```

Requires a running Neo4j instance on bolt://localhost:7687 with the credentials in .env.example.

Step 6. Start backend and frontend

Backend in one terminal.

```
uvicorn backend.main:app --port 8000
```

Frontend in another terminal.

```
cd SystemCode/frontend && npm run dev
```

Open <http://localhost:5173> in Chrome.

C.3 Loading the Chrome extension

1. Open Chrome and go to <chrome://extensions>.
2. Toggle Developer mode in the top right.
3. Click Load unpacked and select the SystemCode/extension/ directory.
4. The PropertyLens icon appears in the extension bar.

C.4 First time setup verification

After step 6, hit <http://localhost:8000/api/model-meta> in a browser. The endpoint returns the loaded model bundle version, the SHA of the artefacts directory, and the timestamps each artefact was last modified. If this returns a JSON object, the system is correctly initialised. If it returns 500, check that step 4 completed without error.

C.5 User guide, one common workflows

C.5.1 Buyer workflow, valuing a single flat

- 5. Navigate to a PropertyGuru HDB listing (any URL matching propertyguru.com.sg/listing/...).
- 6. Click the PropertyLens extension icon. The overlay button appears.
- 7. Click Open in Buyer view. The browser opens a new PropertyLens tab with the eight listing fields pre filled.
- 8. The Buyer view renders the AI fair value, the prediction band, the SHAP driver chart, the comparable past sales table, the offer planner, and the validate listing cards.
- 9. Optionally upload an interior photo to refine the estimate by up to plus or minus 10%.

C.6 Troubleshooting

Symptom	Likely cause	Fix
500 from /api/predict on first request	Artefacts not pulled	Re run step 4
Ask AI returns no LLM available	Ollama not running or Gemma3 not pulled	Run ollama serve and ollama pull gemma3
Chrome extension overlay does not appear on PropertyGuru	Extension not loaded or wrong directory	Reload extension from chrome://extensions
Photo condition score not returned	EfficientNet B0 weights not in artefacts directory	Re run step 4 with --include photo_cnn
Frontend cannot reach backend	CORS or port mismatch	Confirm backend on port 8000 and frontend on 5173. Check .env for VITE_API_URL.
Neo4j seed script fails	Neo4j not running or credentials wrong	Start Neo4j Desktop, verify bolt://localhost:7687, check credentials in .env

Appendix D. Chatbot Evaluation, full per category trace

Automated UI testing done using Claude web browser extension.

#	Input Query	Purpose / Capability tested	Full Output	Status
1	Hi	Greeting / instant response (no LLM call)	"Hi — I can help you search for HDB flats, estimate a listing price, or explain price drivers. Try: "Find a 4-room in Bishan under \$900k near MRT".	✓ PASS
2	Find me a 4-room flat in Bishan under \$900k near MRT	Natural-language property search with weighted scoring	Historical comparables matching your criteria Past resale transactions, most recent sale per address — not current listings. 1. 151 BISHAN ST 11 — sold 2020 for \$420,000 2. 25 SIN MING RD — sold 2025 for \$495,000 3. 170 BISHAN ST 13 — sold 2024 for \$530,000 4. 143 BISHAN ST 12 — sold 2021 for \$568,000	✓ PASS

#	Input Query	Purpose / Capability tested	Full Output	Status
			5. 106 BISHAN ST 12 — sold 2026 for \$608,000 LLM narrative: <i>Based on historical sales, the highest price for a 4-room flat in Bishan was \$608,000 sold in 2024 at 106 Bishan St 12, with other recent sales including \$568,000 in 2024 at 143 Bishan St 12 and \$530,000 in 2024 at 170 Bishan St 13. These sales occurred across addresses like 151 Bishan St 11 and 25 Sin Ming Rd, all within a range under \$900,000.</i> Inferred params: <pre>{"weights": {"score_mrt": 9, "score_value": 7}, "filters": {"flat_type": "4 ROOM", "town": "BISHAN", "max_resale_price": 900000}, "special_query_type": null}</pre>	
3	Show me properties near NANYANG PRIMARY SCHOOL	Knowledge-graph traversal · :NEAR_FAMOUS_SCHOOL edge	Historical comparables near NANYANG PRIMARY SCHOOL <i>Past resale transactions, most recent sale per address — not current listings.</i> 4 QUEEN'S RD — last sale \$460,000 2 QUEEN'S RD — last sale \$660,000 6 FARRER RD — last sale \$620,000 1 QUEEN'S RD — last sale \$900,888 3 QUEEN'S RD — last sale \$670,000 LLM narrative: <i>Here are some recent comparable sales near Nanyang Primary School. In 2024, a 3-room unit at 4 Queen's Rd sold for \$460,000, while a 4-room unit at 1 Queen's Rd sold for \$900,888 in the same year. Additionally, a 4-room unit at 6 Farrer Rd sold for \$620,000 in 2024 and a 4-room unit at 3 Queen's Rd sold for \$670,000 in 2024.</i>	✓ PASS
3a	Show me properties near POI CHING SCHOOL	Shortlist + KG traversal composition	From your shortlist near POI CHING SCHOOL 864 TAMPINES ST 83 · TAMPINES · 4 ROOM · asking \$816,666 · AI \$685,200 · 0.3 km from POI CHING SCHOOL Historical comparables near POI CHING SCHOOL <i>Past resale transactions, most recent sale per address — not current listings.</i> 725 TAMPINES ST 71 — last sale \$824,888 724 TAMPINES ST 71 — last sale \$840,888 726 TAMPINES ST 71 — last sale \$688,000 732 TAMPINES ST 71 — last sale \$751,888 734 TAMPINES ST 71 — last sale \$768,000	✓ PASS

#	Input Query	Purpose / Capability tested	Full Output	Status
			LLM narrative: <i>The closest comparable sales to POI CHING SCHOOL are at 725 Tampines St 71 (sold in 2024 for \$824,888), 724 Tampines St 71 (sold in 2024 for \$840,888), and 726 Tampines St 71 (sold in 2024 for \$688,000). Additionally, properties at 732 Tampines St 71 (sold in 2024 for \$751,888) and 734 Tampines St 71 (sold in 2024 for \$768,000) are also located nearby.</i>	
4	Where can I buy a 4-room in Bedok right now?	<i>Honest framing — out-of-data redirect to live portals</i>	Historical comparables matching your criteria <i>Past resale transactions, most recent sale per address — not current listings.</i> 151 BISHAN ST 11 — sold 2020 for \$420,000 25 SIN MING RD — sold 2025 for \$495,000 170 BISHAN ST 13 — sold 2024 for \$530,000 143 BISHAN ST 12 — sold 2021 for \$568,000 147 BISHAN ST 11 — sold 2025 for \$565,000 LLM narrative: <i>I'm sorry, but the HDB resale registry does not contain any sales data for Bedok. The available data includes sales from Bishan addresses like 151 Bishan St 11 (sold in 2024 for \$420,000), 25 Sin Ming Rd (sold in 2024 for \$495,000), 170 Bishan St 13 (sold in 2024 for \$530,000), 143 Bishan St 12 (sold in 2024 for \$568,000), and 147 Bishan St 11 (sold in 2024 for \$565,000). Please consult PropertyGuru or 99.co for current listings.</i>	✓ PASS
5	Predict the price for BLK 864 Tampines Street 83, 4 room, 122 sqm, floor 20, lease commenced 1987	<i>Tool dispatch · /api/predict (no RAG)</i>	Price estimate Hybrid model predicted SGD 630,000 (approx range 573,314–686,686). Test RMSE ≈ \$37,791.	✓ PASS
6	Show me 5 similar past sales for BLK 864 Tampines Street 83, 4 room, 122 sqm	<i>Tool dispatch · /api/cbr/similar (BallTree)</i>	Similar past sales - TAMPINES BLK 881 TAMPINES ST 84 4 ROOM \$670,000 2026 sim≈2.0% - TAMPINES BLK 862 TAMPINES ST 83 4 ROOM \$680,000 2026 sim≈2.0% - TAMPINES BLK 859 TAMPINES AVE 5 4 ROOM \$675,000 2026 sim≈2.0% - TAMPINES BLK 106 SIMEI ST 1 4 ROOM \$710,000 2026 sim≈2.0% - TAMPINES BLK 226 TAMPINES ST 23 4 ROOM \$638,000 2026 sim≈2.0% - TAMPINES BLK 279 TAMPINES ST 22 4 ROOM \$550,000 2026 sim≈2.0% - TAMPINES BLK 212 TAMPINES ST 23 4 ROOM \$618,000 2026 sim≈2.0% - TAMPINES BLK 868 TAMPINES ST 83 4 ROOM \$650,000 2026 sim≈2.0%	✓ PASS

#	Input Query	Purpose / Capability tested	Full Output	Status
7	Why is BLK 864 Tampines Street 83 priced that way? 4 room, 122 sqm, floor 20	<i>Tool dispatch · /api/explain/composite-shap</i>	SHAP drivers floor_area_sqm: value=122.0 SHAP=+114,297.32 transaction_year: value=2026.0 SHAP=+95,072.00 lease_remaining_years: value=60.0 SHAP=-24,091.67 town_TAMPINES: value=1.0 SHAP=+16,117.60 mall_weighted_access_3km: value=3.9353 SHAP=-14,827.07 room_count: value=4.0 SHAP=+12,622.23 flat_model_Model A: value=0.0 SHAP=-6,294.69 level_mid: value=8.0 SHAP=+4,806.09 town_BISHAN: value=0.0 SHAP=-4,144.05 flat_model_Improved: value=1.0 SHAP=-3,767.57 dist_to_nearest_mall_m: value=1001.2328 SHAP=-3,765.47 flat_type_3 ROOM: value=0.0 SHAP=+2,630.68	✓ PASS
8	my shortlist	<i>Wishlist read · graph view with vs-historical comparison</i>	Your shortlist (graph view — vs historical sales) 1. 864 TAMPINES ST 83 · TAMPINES · 4 ROOM · asking \$816,666 · AI \$685,200 · hist \$828,000 (2026) (-1.4% vs hist) · nearest famous school: POI CHING SCHOOL (0.3 km) 2. 629 HOUGANG AVE 8 · HOUGANG · 3 ROOM · asking \$430,000 · AI \$407,900 · hist \$420,000 (2025) (+2.4% vs hist) · nearest famous school: ROSYTH SCHOOL (0.8 km) 3. 242 BISHAN ST 22 · BISHAN · 4 ROOM · asking \$880,000 · AI \$786,800 · hist \$1,050,000 (2025) (-16.2% vs hist) · nearest famous school: AI TONG SCHOOL (1.3 km) 4. 578 HOUGANG AVE 4 · HOUGANG · 4 ROOM · asking \$1,150,000 · AI \$835,300 · hist \$860,000 (2025) (+33.7% vs hist) · nearest famous school: ROSYTH SCHOOL (1.3 km) 5. 324 SERANGOON AVE 3 · SERANGOON · 4 ROOM · asking \$785,000 · AI \$776,500 · hist \$751,000 (2025) (+4.5% vs hist) · nearest famous school: PEI CHUN PUBLIC SCHOOL (2.2 km) 6. 1 PINNACLE @ DUXTON · CENTRAL AREA · 4 ROOM · asking \$1,300,000 · AI \$1,020,300 · (no historical sale on record) <i>1 of 6 saves are at addresses with no historical sale data — only town-level comparison available for those.</i>	✓ PASS
9	my saves by town vs historical median	<i>Wishlist aggregation · per-town summary stats</i>	Your saves by town vs historical median [object Object]	✓ PASS
10	Predict the price for the latest flat in my shortlist	<i>Wishlist + tool composition · auto-resolve flat from saves</i>	Price estimate Hybrid model predicted SGD 776,500 (approx range 719,814–833,186). Test RMSE ≈ \$37,791. <i>Auto-resolved from wishlist to 324 Serangoon Ave 3 (most recent save).</i>	✓ PASS

Appendix E: References

E.1 Academic papers

The PropertyLens system design draws on five threads of academic literature.

Hybrid cluster stack architectures

- Sun, Y., Liu, Y., and Wang, X. (2024). A Hybrid Machine Learning Model Architecture with Clustering Analysis and Stacking Ensemble for Real Estate Price Prediction. ResearchGate publication 383237853. Available at researchgate.net/publication/383237853.

Explainable AI

- Lundberg, S. M., and Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30 (NeurIPS), 4765 to 4774. The original SHAP paper.
- Lundberg, S. M., et al. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. Nature Machine Intelligence 2 (1), 56 to 67. The TreeSHAP exact algorithm paper.
- Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). Why Should I Trust You? Explaining the Predictions of Any Classifier. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1135 to 1144. The original LIME paper.
- Aamodt, A., and Plaza, E. (1994). Case Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches. AI Communications 7 (1), 39 to 59. The foundational CBR formalisation.

Tree based ensembles

- Chen, T., and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785 to 794.
- Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems 30 (NeurIPS), 3146 to 3154.
- Breiman, L. (2001). Random Forests. Machine Learning 45 (1), 5 to 32. The original Random Forest paper.

Convolutional networks for image regression

- Tan, M., and Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. Proceedings of the 36th International Conference on Machine Learning (ICML), 6105 to 6114.
- Schuhmann, C., et al. (2022). LAION 5B: An Open Large Scale Dataset for Training Next Generation Image Text Models. Advances in Neural Information Processing Systems 35 (NeurIPS Datasets and Benchmarks Track). The aesthetic predictor used to label LSUN bedrooms for the photo condition CNN.

Knowledge graphs and retrieval augmented generation

- Lewis, P., et al. (2020). Retrieval Augmented Generation for Knowledge Intensive NLP Tasks. Advances in Neural Information Processing Systems 33 (NeurIPS), 9459 to 9474. The original RAG paper.
- Edge, D., et al. (2024). From Local to Global: A Graph RAG Approach to Query Focused Summarisation. Microsoft Research technical report. arxiv.org/abs/2404.16130. The graph RAG approach informing PropertyLens's Ask AI design.

E.2 Singapore data and government sources

- **data.gov.sg** Resale Flat Prices dataset. data.gov.sg/dataset/resale-flat-prices. Source for all HDB resale transaction history used in training and evaluation.

- **OneMap API** Address geocoding and MRT/LRT station coordinates. onemap.gov.sg/apidocs/.
- **Ministry of Education, Singapore** School master dataset. moe.gov.sg/schoolfinder. Source for the famous school tier ranking.
- **Singapore Department of Statistics** Population and household statistics. singstat.gov.sg. Source for the 77% HDB residency figure cited in Section 1.1.
- **ERA Singapore** 4Q 2025 HDB Quarterly Report. era.com.sg. Source for the 26,169 transaction figure for 2025 cited in Section 1.1.
- **OpenStreetMap** Highway and major road geometry for the noise proximity proxy feature. openstreetmap.org.

E.3 Software libraries and tools

- **scikit learn** Pedregosa, F., et al. (2011). Machine Learning in Python. Journal of Machine Learning Research 12, 2825 to 2830. scikit-learn.org.
- **SHAP library** github.com/shap/shap. Used for TreeExplainer in Composite TreeSHAP.
- **FastAPI** fastapi.tiangolo.com. Backend web framework.
- **React 18 with Vite** react.dev and vitejs.dev. Frontend framework and build tool.
- **Tailwind CSS** tailwindcss.com. Frontend styling framework.
- **Neo4j** neo4j.com. Graph database backing the Ask AI knowledge graph.
- **Ollama** ollama.ai. Local LLM serving framework wrapping Gemma3.
- **Hugging Face Hub** huggingface.co. Hosts trained model bundles, photo CNN weights, frozen datasets, and the LSUN bedroom dataset.
- **LSUN bedrooms dataset** huggingface.co/datasets/pcuenq/lsun-bedrooms. Source images for the photo condition CNN training set.

E.4 Comparable commercial products surveyed

- **PropertyGuru Singapore** propertyguru.com.sg. Listing portal. PropertyLens Chrome extension bridges listings from PropertyGuru into the valuation flow.
- **99.co Singapore** 99.co. Listing portal.
- **SRX X Value** srx.com.sg. AI valuation product for Singapore residential property.
- **HomerAI** homeraai.com. Conversational property assistant.

Appendix F. Composite TreeSHAP held-out validation

F.1 Aim

Quantitatively evaluate the explainability layer of the Hybrid Cluster Ensemble. Four properties are checked per case: (i) additivity of the SHAP attributions to the hybrid prediction, (ii) determinism of the explainer, (iii) domain plausibility of attribution signs against cluster-mean priors, and (iv) CBR neighbourhood quality for the comparables surfaced alongside each explanation.

F.2 Sample design

A stratified sample of 100 transactions was drawn from the temporal hold-out test split, targeting ≥ 6 cases per (cluster $\times$ flat-type) cell. The realised sample composition:

Stratum	Distribution
Flat type	3-ROOM 24 · 4-ROOM 31 · 5-ROOM 26 · EXECUTIVE 19
Cluster	C0 34 · C1 46 · C2 20
Year	2023 28 · 2024 30 · 2025 38 · 2026 4
Towns	24 distinct towns covered

F.3 Headline results

Check	Pass rate	Notes
Additivity (ϕ -sum reconstructs tree-stack prediction at $\leq 0.1\%$ tolerance)	100 / 100	Mean error $3.6 \times 10^{-7} \%$, max $1.4 \times 10^{-6} \%$. Numerically tight; confirms the per-learner TreeSHAP outputs blended through cluster meta-weights reconstruct <code>composite_prediction</code> exactly.
Determinism (identical input $\rightarrow$ identical attributions)	100 / 100	Expected — TreeSHAP is deterministic. Documented to catch any silent regression to a sampling explainer.
Domain plausibility (sign of ϕ matches direction of feature value vs. cluster mean for monotone features)	95 / 100	All 5 fails are local cluster sub-market effects, not bugs (see F.5).
CBR same-town (k=5 comparables, all from the listing's town)	100 / 100	Fallback to adjacent towns never triggered in this sample. Median same-town count = 5/5.

F.4 Ridge reconciliation gap

The Ridge linear component of the meta-learner is excluded from the TreeSHAP additivity sum by design and surfaced as `approximation_error` on `/api/explain/composite-shap`. Across the 100 cases:

Statistic	Value
Min	0.03%
Median	0.07%
Mean	0.07%
Max	0.18%
Stdev	0.04%

Per-cluster means: C0 0.04%, C1 0.07%, C2 0.15%. Cluster 2 carries a slightly larger gap, consistent with its Ridge meta-weight being heavier in the bundle. All values fall well below the 5–7% envelope cited in the system design — the deployed model is reconstructed almost entirely by the tree stack, which is what makes Composite TreeSHAP a faithful explainer for it.

F.5 Plausibility violations (5 / 100)

#	Cluster	Flat type	Town	Year	Violation
36	1	5-ROOM	PUNGGOL	2024	<code>floor_area</code> 110 sqm > C1 mean 107.9 but $\phi = -19,207$; <code>lease</code> 79 yr > C1 mean 77.3 but $\phi = -21,507$
37	1	5-ROOM	PUNGGOL	2025	<code>floor_area</code> 110 sqm > C1 mean 107.9 but $\phi = -20,154$
40	1	5-ROOM	WOODLANDS	2025	<code>floor_area</code> 110 sqm > C1 mean 107.9 but $\phi = -13,853$

56	2	4-ROOM	KALLANG/WHAMPOA	2025	lease 70 yr > C2 mean 68.3 but $\phi = -51,588$
83	1	4-ROOM	WOODLANDS	2025	lease 78 yr > C1 mean 77.3 but $\phi = -22,464$

These are not violations of additivity or determinism - they are cases where the cluster stack overrode a global prior to capture a local sub-market effect (e.g. 110 sqm 5-ROOMs in Punggol/Woodlands selling below the cluster norm; near-median leases in Kallang/Whampoa carrying a discount because that cluster's leased-up flats trade well above the mean). Per-cluster pass rates (C0 34/34, C1 42/46, C2 19/20) confirm the dispersion is in the clusters designed to absorb the most local pricing dynamics, which is the intended behaviour.

F.6 Driver-feature stability

Frequency with which each feature appeared in a case's top-6 SHAP drivers:

Feature	Top-6 frequency
transaction_year	100 / 100
floor_area_sqm	97
room_count	93
lease_remaining_years	82
level_mid	58
dist_to_highway_m	46
mall_weighted_access_3km	32
dist_to_foodcourt_m	15
mall_count_3km	14
dist_to_nearest_mall_m	12
dist_to_mrt_m	12
flat_type_5 ROOM	11

The four global structural drivers (year, area, room count, remaining lease) appear in $\geq 82\%$ of cases. The next tier — storey, highway distance, mall access — surfaces in roughly half, showing the explainer is not collapsed onto a single dominant feature but does converge on a stable HDB-pricing narrative across cases.

F.7 Take-aways for the report

1. The tree-stack portion of the Hybrid Cluster Ensemble is exactly explainable: SHAP additivity holds to numerical precision on every case.
2. The $\sim 0.07\%$ median Ridge gap means the linear corrector almost never moves the prediction enough to bias the visual narrative — the chart the user sees on /buyer and /seller faithfully represents the deployed model.
3. The 5 plausibility “fails” are all in the same direction (a feature higher than the cluster mean carrying a negative attribution) and concentrated in C1/C2 — consistent with the design goal of allowing cluster sub-models to capture local price dynamics rather than enforcing a global monotone prior.
4. CBR retrieval was sufficient on its primary path for every test case, with no recourse to the adjacency fallback in this sample.

Appendix G. Pinecone Vector RAG, exploratory work

Section 4.7.4 mentions the team built and evaluated a Pinecone vector RAG implementation as an alternative to the Neo4j knowledge graph approach. This appendix documents that exploratory work, why it was built, why it was rejected, and what was preserved in the codebase for traceability.

G.1 What was built

A complete vector RAG pipeline was implemented in parallel with the knowledge graph approach. Four components.

- **Embedding generation.** Each Property node embedded with sentence transformer all-MiniLM-L6-v2. Text input: structured fields rendered as a sentence (e.g. "4 ROOM flat in Bishan with 95 sqm floor area, 75 years lease remaining, 280m to Bishan MRT, sold March 2024 for \$920,000"). 384 dimensions, unit-normalised.
- **Vector index.** Uploaded to a Pinecone serverless index (us-east-1). Each vector stored with metadata (town, flat type, floor area, price, year) for filter pre-selection.
- **Retrieval.** User query embedded via the same transformer. Pinecone returns top K nearest vectors, optionally filtered by metadata. Results feed an LLM narrative synthesis stage equivalent to Stage 3 of the KG pipeline.
- **Evaluation harness.** The same 25-case evaluation set used for the knowledge graph chatbot.

G.2 Why it was rejected

The Pinecone pipeline produced reasonable average-case results. Three concrete reasons drove the rejection.

- No auditability. With vector retrieval the user cannot see why a specific property was returned. The embedding model judged it similar but the dimensions of similarity are opaque. With Cypher, inferred parameters and graph traversal are inspectable.
- Brittle on negative constraints. Positive constraints ("near MRT") embed cleanly. Negative constraints ("not central", "not above \$500,000") do not. 3 of the 5 Category 6 impossible queries returned plausible-looking but incorrect results, where the KG correctly returned zero with an explanation.
- Extra infrastructure dependency. Pinecone is hosted, with API rate limits and per-query cost. Neo4j runs locally. For a project prioritising reproducibility, a service requiring an account and API key adds friction without architectural benefit.

G.3 Comparative evaluation

The same 25-case evaluation was run against both pipelines. KG reached 80%, Pinecone reached 60%.

Category	Knowledge graph	Vector RAG	Difference
Cat 1, MRT plus budget	75% (3 of 4)	75% (3 of 4)	Tied
Cat 2, school plus budget	50% (2 of 4)	25% (1 of 4)	KG ahead
Cat 3, quiet plus size plus lease	100% (4 of 4)	75% (3 of 4)	KG ahead
Cat 4, MRT plus size plus budget	33% (1 of 3)	33% (1 of 3)	Tied
Cat 5, near specific famous school (graph traversal)	100% (5 of 5)	60% (3 of 5)	KG decisively ahead
Cat 6, intentionally impossible queries	100% (5 of 5)	40% (2 of 5)	KG decisively ahead

Total	80% (20 of 25)	60% (15 of 25)	20 percentage points
-------	----------------	----------------	----------------------

The decisive gap is in Category 5 (graph traversal queries about specific famous schools) and Category 6 (intentionally impossible queries). Vector RAG cannot follow a typed edge from a school node to its near properties as a structured operation. Vector RAG cannot reliably recognise that no flat in Bishan transacts under \$100,000 because the embedding for the impossible query still returns the closest available matches. Both behaviours are critical for a buyer making a real purchase decision and both are correctly handled by the knowledge graph.

G.4 Preserved artefacts

The exploratory work is preserved at `SystemCode/notebooks/04_xai_layer/06b_kernel_shap_explain.ipynb` (the kernel SHAP comparison) and `property_search_rag.py` retains a deprecated branch for the vector path. Neither is exposed in the production frontend. They are kept for traceability so future iterations can revisit the comparison.

When vector RAG would have been the better choice. Truly open-ended queries that do not map cleanly onto the curated graph schema (unstructured user reviews, free-text neighbourhood descriptions, photo captions). PropertyLens does not have that requirement. Every PropertyLens query reduces to structured filters and weights against typed nodes, which is exactly the case Cypher handles best.